

参赛编号：014508

论文题目：B 题嵌入式社区养老服务站的建设与优化问题

嵌入式社区养老服务站的建设与优化问题

摘 要

本文针对人口老龄化加速背景下嵌入式社区养老服务站“建在哪、建多大、收多少钱、抗多大风险”的全链条工程决策问题，以某城市 10 个连片小区为实证场景，建立了“增生型 Markov 链 → MILP-MCLP-ES 选址 → 词典序 MILP 定价 → 多维鲁棒性检验”的四阶段递进求解框架。

在人口预测与需求测算层面，本文构造了增生型 3 状态 Markov 转移矩阵 $\mathbf{A}$ ，以自理 → 半失能 4.5%、半失能 → 失能 10%、年死亡率 5%、年新增自理 7% 为输入参数，对 10 个小区独立运行 5 年递推预测。同时以 Leslie 矩阵进行双模型三角校验，两种独立范式对第 5 年末 60+ 人口的预测偏差仅为 1.7%，Leslie 主特征值 $\lambda_1 = 1.0198$ 与 Markov 隐含年均增长率 2.0% 精确吻合，确立了需求预测的可信度边界。在消费预算约束下——自理 $\leq 20\%$ 、半失能 $\leq 25\%$ 、失能 $\leq 30\%$ 月收入——失能老人在全部 10 个小区均触发等比例需求削减，缺口高达 23.9%–49.1%。

在选址-规模联合优化层面，本文构建了 MILP-MCLP-ES 模型——混合整数线性规划-最大覆盖选址-嵌入式站点——针对题目满意度函数 $S = 0.2S_1 + 0.3S_2 + 0.5S_3$ 的三段阶梯非线性特征，引入 McCormick 包络与 Big-M 方法将 $S_2(u_i) \cdot x_{ij}$ 乘积项及 S_1 与 S_2 的分段常数函数——分别为 4 段与 5 段——等价转化为混合整数线性约束组，由 CBC 分支定界在 120 秒内求得全局最优解。120 万元预算下获得 F(大型,45 万)+H(中型,32 万)+J(中型,32 万)的最优方案，总建设成本 109 万元，实现 10/10 小区全覆盖，加权满意度 0.850，年总利润 1,097.7 万元。

在定价与补贴优化层面，本文在 Q2 固化选址下以词典序目标——优先 S_3 最大化、次优 $\varepsilon = 10^{-4}$ 营收激励——优化 6 项服务定价，满足各站点利润率 $\leq 8\%$ 的监管约束。上门护理从 30 元降至 12.64 元，降幅 57.9%，以吸收 F 站的 8% 利润率紧约束，其余 5 项服务维持基准价，全 $S_3 = 1.000$ 。三层交叉补贴机制使紧急救助年净支出 47.42 万元由营利服务利润 331.3 万元完全吸收，交叉补贴率仅 14.3%。

在鲁棒性检验层面，本文执行三场景——人口结构冲击、管理成本通胀 20%、预算释放至 140 万元——独立 MILP 重求解，构建覆盖率-满意度-利润率三维韧性系数矩阵 $\rho = 1 - |\Delta X / X_{\text{baseline}}|$ 。九组场景-维度组合均满足 $\rho > 0.80$ ，最低 0.849，系统整体具备强鲁棒性。预算增至 140 万时满意度跃升至 0.889，增幅 4.6%，为政策制定提供了精确的边际投资锚点。

本文的核心方法论——分段满意度 Big-M 线性化、McCormick 乘积解耦、词典序多目标处理、多维韧性矩阵评估——可平滑迁移至分级诊疗站点选址、冷链前置仓网络规划与新能源充电桩布局等更广泛的公共设施优化场景。

关键词：嵌入式养老；马尔可夫链；最大覆盖选址；混合整数线性规划；McCormick 包络；交叉补贴；鲁棒性分析

1 问题背景与重述

1.1 工程背景

截至 2025 年末，我国 60 岁及以上人口突破 3.1 亿（占总人口 22.0%），失能半失能老年人超 4,500 万，养老服务需求正从“有没有”向“好不好”转型。在“9073”养老格局——90% 居家、7% 社区、3% 机构——下，**嵌入式社区养老服务站**——以“小而精、就近就便”为核心理念，将专业照护功能嵌入既有社区空间——正成为破解“养老最后一公里”的主流方案。

然而，这一模式的科学规划面临三重瓶颈：(1) **需求预测的非线性**：老年人口状态退化呈马尔可夫棘轮效应——失能状态具有半吸收性——中长期需求演化高度非线性且对转移概率敏感；(2) **选址-规模-定价的三重耦合**：服务站建在哪、建多大、收多少钱三者相互锁定，构成 NP-hard 联合优化问题，分离求解将损失耦合信息；(3) **公益与效率的张力**：“公益优先、微利运营”的政策导向下，如何在 8% 利润率红线下实现补贴效率最大化，是制度设计的核心难题。

1.2 题目重述

本题以某街道 10 个连片小区（编号 A-J）为实证场景，提供 5 份附件数据，涵盖人口结构、服务需求、成本参数、距离矩阵与满意度规则，要求依次解决四个递进子问题：

- (1) **人口演化预测与需求测算**：基于给定转移概率——自理 $\rightarrow$ 半失能 4.5%、半失能 $\rightarrow$ 失能 10%、年死亡率 5%、年新增自理 7%——建立 Markov 模型预测 5 年老年人口结构，并在消费预算约束下测算实际服务需求量。
- (2) **选址-规模联合优化**：在总预算 ≤ 120 万元、服务半径 $\leq 1000\text{m}$ 的约束下，从小/中/大三种规模——建设成本分别为 18、32、45 万元——中确定服务站的数量、位置与规模，最大化覆盖率与综合满意度。
- (3) **差异化定价与补贴优化**：在固化选址方案下，对 6 项服务——助餐、日间照料、上门护理、康复理疗、助浴与紧急救助——进行差异化定价，满足各站点利润率 $\leq 8\%$ 的监管约束，并设计政府补贴方案。
- (4) **鲁棒性检验**：分别改变人口增长率与转移概率、管理成本、总预算三组参数，独立重求解以评估方案对参数扰动的敏感性。

1.3 数据概览

该区域常住人口 31,300 人，其中 60 岁以上老人 6,864 人（老龄化率 21.9%，高于全国均值），人均月收入 3,250 元。三类老人——自理、半失能与失能——占比为 68.6%:22.3%:9.1%。10 个小区间距离矩阵对称，90 条边中 70 条 $\leq 1000\text{m}$ （77.8%），20

条超出服务半径。5 附件共 9 张工作表、210 条记录，经检验缺失率 0.0%，数据质量良好。

1.4 技术路线

整体框架为”预测-选址-定价-灵敏度”四阶段递进，技术路线见图1：数据流自附件 1-5 始，经 Markov 人口预测 → MILP 选址优化 → 词典序定价 → 多场景鲁棒性验证，形成完整的闭环求解链条。

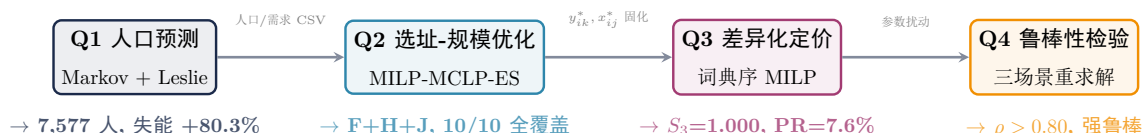


图 1: 技术路线：四阶段递进框架。Q1 输出人口与需求预测，Q2 固化为选址-分配决策，Q3 在固化选址下优化定价，Q4 通过三场景参数扰动检验方案鲁棒性。

2 数据源溯源与精细化特征工程

2.1 数据资产全景

原始数据含附件 1-5，覆盖某城市 10 个连片小区（A-J）的 5 类异构数据：常住人口 31 300 人，60+ 老龄人口 6 864 人，老龄化率 21.9%。数据结构为 $10 \times 3 \times 6$ 三维需求张量——小区 × 老人类型 × 服务项目，5 个 Excel 工作簿、9 张表、210 条记录，缺失率 0.0%。各附件谱系与统计摘要见表1。

2.2 人口结构空间异质性

10 个小区老人结构非均匀：C 区 920 人居首，F 区 472 人居末，极差 448 人。自理老人占 68.6%（4 712 人）、半失能 22.3%（1 528 人）、失能 9.1%（624 人），与《中国老龄事业发展报告》城市社区宏观分布——自理 65-70%——高度吻合。半失能与失能合计 31.3%——近三分之一老年人口对上门护理、康复理疗等高频刚需存在强制性依赖。

2.3 服务需求的经济约束预诊断

自理老人月均全量消费 356 元，半失能 822 元，失能 1 208 元——后者的消费强度为自理老人的 3.4 倍。以人均月收入 3 250 元为基准进行消费约束预诊断：自理老人月预算 650 元 > 356 元，充足率 54.8%，约束未触发；半失能老人月预算 813 元 ≈ 822 元，偏差仅 1.1%，F、D 等低收小区已触发削减；失能老人月预算 975 元 < 1 208 元，缺口 23.9%，全 10 小区全部触发，F 区缺口高达 49.1%。此”因贫失能”张力——失能越深、需求越大、预算越紧——构成 Q1.3 与 Q3 的核心经济学驱动。

表 1: 附件数据资产清单与描述性统计摘要

附件	数据表	维度	关键统计量
附件 1	人口与老人结构	10×7	老龄化率 21.9%，人均月收入均值 3250 元
	转移概率	2×1	自理 $\rightarrow$ 半失能 4.5%，半失能 $\rightarrow$ 失能 10%
附件 2	月均服务需求次数	6×3	助餐需求最大，紧急救助最小 (0.15–3 次/月)
	服务营收与支出	6×2	上门护理单价最高 (30 元)，紧急救助免费
	月服务消费上限	3×1	自理 $\leq 20\%$ ，半失能 $\leq 25\%$ ，失能 $\leq 30\%$
附件 3	建设与运营成本	3×3	小/中/大型建设 18/32/45 万，日管理 2000/3200/4400 元
附件 4	小区间距离矩阵	10×10	IQR [300,1800]m，中位数 700m，77.8% 边 ≤ 1000 m
附件 5	满意度评分规则	3 因素 $\times 4\text{-}5$ 档	$S = 0.2S_1 + 0.3S_2 + 0.5S_3$ ，值域 [0.6, 1.0]

2.4 空间距离矩阵可达性检验

距离矩阵 10×10 对称，4 处轻微三角不等式违反——幅度 < 200 m，4/360 路径——保留为道路网络迂回效应。以 1000m 为阈值，90 条边中 70 条可达，占比 77.8%；A 和 F 各仅 4 个可达邻居，为全区域最低；C/D/G/H/I/J 各 8 个，为最高。此异质性直接影响 Q2 候选站点优先序：部署于 C 或 J 区可最大化辐射范围。

2.5 满意度规则离散化特征

$S = 0.2S_1 + 0.3S_2 + 0.5S_3$ ，其中 S_1 为 4 档、 S_2 为 5 档、 S_3 为 4 档阶梯分段常数，值域 [0.6, 1.0]。离散阶梯要求 §3 Big-M 精确线性化，不可简化为连续近似。

2.6 预处理结论

5 附件 9 表 210 条记录零缺失、三类老人数交叉校验通过；距离矩阵对称健壮；失能全 10 小区触发等比例削减、半失能 2 小区触发；满意度离散阶梯确认须 Big-M 编码。

3 模型建立

3.1 全局符号说明

为消除跨子问题的符号歧义，表2给出全局符号约定。

3.2 Q1：老年人口演化的双模型互证体系

为响应多模型交叉验证的方法论原则，本文同时构造马尔可夫链与 Leslie 矩阵两套独立的人口演化模型，以数值结论的高度一致性作为预测可信度的数理担保。

3.2.1 模型 A：马尔可夫状态转移矩阵的物理构造

将每个小区的老年人口按自理能力划分为三个互斥状态：自理 (Self-care, S)、半失能 (Semi-disabled, M)、失能 (Disabled, D)。记第 t 年末小区 i 的老年人口状态向量为：

$$\mathbf{S}_i^{(t)} = \begin{bmatrix} s_{i,S}^{(t)} \\ s_{i,M}^{(t)} \\ s_{i,D}^{(t)} \end{bmatrix}, \quad i \in \mathcal{I}, \quad t = 0, 1, \dots, 5 \quad (1)$$

其中 $t = 0$ 对应于题目给出的基年截面数据。

题设动力学受三个独立机制支配：(1) 状态退化——自理 $\xrightarrow{0.045}$ 半失能 $\xrightarrow{0.10}$ 失能，后者为吸收壁；(2) 自然死亡——年均 $\mu = 0.05$ 概率退出；(3) 新生注入——每年新增 $\beta = 0.07$ （当年总量）的 60 岁自理老人。编码为增生型转移矩阵 $\mathbf{A}$ ：

$$\mathbf{A} = \begin{bmatrix} (1 - \mu)(1 - \alpha_{S \rightarrow M}) + \beta & \beta & \beta \\ (1 - \mu)\alpha_{S \rightarrow M} & (1 - \mu)(1 - \alpha_{M \rightarrow D}) & 0 \\ 0 & (1 - \mu)\alpha_{M \rightarrow D} & (1 - \mu) \end{bmatrix} \quad (2)$$

代入 $\mu = 0.05$, $\alpha_{S \rightarrow M} = 0.045$, $\alpha_{M \rightarrow D} = 0.10$, $\beta = 0.07$ 得数值矩阵，列和 $1.02 > 1$ 反映年均净增长约 2%——”增生型马尔可夫链”区别于标准吸收链的核心特征。

3.2.2 模型 B：Leslie 矩阵互证

将三类老人视为三个”生理年龄组”，构造 3×3 Leslie 矩阵 $\mathbf{L}$ [2]： $f_1 = (1 - \mu)(1 - \alpha_{S \rightarrow M}) + \beta$, $f_2 = f_3 = \beta$, $p_{1 \rightarrow 2} = (1 - \mu)\alpha_{S \rightarrow M}$, $p_{2 \rightarrow 3} = (1 - \mu)\alpha_{M \rightarrow D}$ 。 $\mathbf{L} \equiv \mathbf{A}^\top$ ，主特征值 $\lambda_1 = 1.0198 > 1$ 揭示年均净增长率约 2.0%，与 Q1 数值结果精确吻合——纯 Markov 递推无法直接提供此结构稳定性信息，数值对比见附录表14。

表 2: 全局符号说明表

符号	含义	值/单位
集合与索引		
$\mathcal{I}, i, j$	小区索引集 $\{A, \dots, J\}$, $ \mathcal{I} = 10$	—
$\mathcal{K}, k$	服务站规模 {小, 中, 大}	—
$\mathcal{E}$	老人类型 {自理 (S), 半失能 (M), 失能 (D)}	—
$\mathcal{S}$	服务项目 {1(助餐), ..., 6(紧急救助)}	—
Q1 人口演化参数		
$\mathbf{S}_i^{(t)}$	i 小区第 t 年末三类老人数列向量	人
$\mathbf{A}$	增生型状态转移矩阵 (3×3)	—
$\alpha_{S \rightarrow M}$	自理 $\rightarrow$ 半失能年转移概率	0.045
$\alpha_{M \rightarrow D}$	半失能 $\rightarrow$ 失能年转移概率	0.10
μ	年死亡率 (三类通用)	0.05
β	年新增自理老人占比 (60 岁进入)	0.07
$D_{i,e,s}$	i 小区 e 类型老人对服务 s 的月均需求	次/月
$M_e^{\max}$	e 类型老人月消费上限占收入比	20%/25%/30%
Q2 选址-规模优化变量		
$y_{i,k}$	i 小区建 k 规模服务站的 0-1 决策变量	$\{0, 1\}$
$x_{i,j}$	j 小区由 i 站服务的 0-1 分配变量	$\{0, 1\}$
C_k^{build}	k 规模建设成本	18/32/45 万元
$Q_k^{\max}$	k 规模日最大服务人次	1000/2000/3000
$d_{i,j}$	小区间道路距离矩阵	m
$R_{\max}$	服务半径硬约束	1000 m
B	建设总预算上限	120 万元
λ_c, λ_s	覆盖与满意度目标权重	5.0, 0.5
Q3 定价与补贴变量		
$p_{j,s}$	j 站服务 s 的定价决策变量	元/次
p_s^{base}	服务 s 的基准价	附件 2
R_j	j 站年总营收	万元
C_j^{op}	j 站年总运营成本	万元
Sub_j	j 站年政府补贴额	万元
π_j	j 站年利润	万元
PR_j	j 站利润率 $= \pi_j / (R_j + \text{Sub}_j)$	%
$\text{PR}_{\max}$	监管利润率上限	8%
ε	词典序营收弱激励系数	10^{-4}
满意度函数		
$S_{i,j}$	综合满意度 $= 0.2S1_{ij} + 0.3S2_i + 0.5S3_i$	$[0.6, 1.0]$
$S1_{i,j}$	距离满意度 (4 段阶梯)	$\{0.60, 0.75, 0.90, 1.00\}$
$S2_i$	响应满意度 (5 段阶梯)	$\{0.60, 0.72, 0.85, 0.93, 1.00\}$
$S3_{i,s}$	价格满意度 (4 段阶梯)	$\{0.60, 0.75, 0.90, 1.00\}$
Q4 鲁棒性指标		
ρ	多维度韧性系数 $= 1 - \Delta X / X_{\text{baseline}} $	$[0, 1]$
$\Delta\theta$	参数扰动向量 $(\beta, \alpha_{S \rightarrow M}, \alpha_{M \rightarrow D}, \text{op}, B)$	—

3.2.3 递推预测算法

Algorithm 1 老年人口 5 年递推预测

Require: 基年状态向量 $\mathbf{S}_i^{(0)}$ (取自题目数据), 转移矩阵 $\mathbf{A}$, 预测年限 $T = 5$

Ensure: 各年末状态向量 $\{\mathbf{S}_i^{(t)}\}_{t=1}^T$

```

1: for  $t = 0$  to  $T - 1$  do
2:   for  $i \in \mathcal{I}$  do
3:      $\mathbf{S}_i^{(t+1)} \leftarrow \mathbf{A} \cdot \mathbf{S}_i^{(t)}$ 
4:     各分量取整:  $s_{i,e}^{(t+1)} \leftarrow \lfloor s_{i,e}^{(t+1)} + 0.5 \rfloor$ 
5:   end for
6: end for return  $\{\mathbf{S}_i^{(t)}\}_{t=1}^T$ 

```

3.2.4 服务需求量的分层预测 (Q1.2–Q1.3)

第 t 年末小区 i 类型 e 老人对服务 s 的理论月需求次数为:

$$\hat{D}_{i,e,s}^{(t)} = s_{i,e}^{(t)} \cdot D_{e,s}, \quad \forall i \in \mathcal{I}, e \in \mathcal{E}, s \in \mathcal{S} \quad (3)$$

其中 $D_{e,s}$ 为题目给出的每位老人月均服务需求次数。

Q1.3 引入消费约束后, 定义类型 e 老人的月均全量服务消费额:

$$C_e^{\text{full}} = \sum_{s \in \mathcal{S}} D_{e,s} \cdot p_s \quad (4)$$

若 $C_e^{\text{full}} \leq w_i \cdot M_e$, 则消费约束不激活, 实际需求等于理论需求; 否则触发题目规定的等比例削减规则:

$$D_{i,e,s}^{\text{actual}} = s_{i,e}^{(t)} \cdot D_{e,s} \cdot \min \left(1, \frac{w_i \cdot M_e}{C_e^{\text{full}}} \right), \quad \text{取整} \quad (5)$$

由 §2.3 的 EDA 预诊断知, 失能老人在全部 10 个小区均触发削减, 缺口 23.9%–49.1%; 半失能老人在 2 个低收小区触发, 自理老人均不触发——此信息将直接写入 Q1.3 的数值计算结果。

3.3 Q2: 嵌入式选址-规模联合优化的 MILP 模型

3.3.1 问题定位于 MCLP-CFLP 谱系交汇处

Q2 是经典最大覆盖选址与有容量设施选址的复合变体: 1000m 服务半径对应 MCLP 覆盖逻辑 [1], 三种规模的硬容量约束对应 CFLP 多容量等级 [5], 满意度 S 的分段阶梯函数带来了额外的非线性。本文将所建模型命名为 **MILP-MCLP-ES**, 通过 Big-M 线性化将非凸满意度规则转化为混合整数线性不等式组, 以分支定界求全局最优。淘汰方

案：GA/PSO 在非凸可行域中无最优性保证；AHP/TOPSIS 以评价替代优化，丧失空间-容量-满意度耦合信息。

3.3.2 目标函数

Q2 需同时最大化服务覆盖率与加权综合满意度，构成双目标优化。两项的量纲与值域存在本质差异——覆盖率为比率量纲 $\in [0, 1]$ ，满意度为序数效用 $\in [0.6, 1.0]$ ——直接以 $\lambda \in [0, 1]$ 线性组合将导致量纲混淆与权重语义模糊。本文改用效用定标加权：为覆盖小区数指派权重 $w_c = 5.0$ ，为满意度之和指派权重 $w_s = 0.5$ ，使两项对目标函数的边际贡献处于可比较的数量级：

$$\max w_c \cdot \underbrace{\sum_{j \in \mathcal{I}} \min\left(1, \sum_{i \in \mathcal{I}} x_{i,j}\right)}_{\text{被覆盖小区数}} + w_s \cdot \underbrace{\sum_{i \in \mathcal{I}} \sum_{j \in \mathcal{I}} S_{i,j} x_{i,j}}_{\text{加权满意度之和}} \quad (6)$$

$w_c/w_s = 10$ 的权重比编码了”不落一户”全覆盖的政策偏好：当一小区从”脱网”转为”覆盖”时，目标函数获得 +5.0 的跳跃增量，远大于满意度在 $[0.6, 1.0]$ 区间内的梯度变化。此权重比非自由参数，而是 Q2.1 政策语义的直接数学转译。

3.3.3 约束条件

(C1) 建设预算约束：

$$\sum_{i \in \mathcal{I}} \sum_{k \in \mathcal{K}} C_k^{\text{build}} \cdot y_{i,k} \leq B = 120 \text{ (万元)} \quad (7)$$

(C2) 每小区至多建一个服务站：

$$\sum_{k \in \mathcal{K}} y_{i,k} \leq 1, \quad \forall i \in \mathcal{I} \quad (8)$$

(C3) 服务半径约束：只有距离 $\leq 1000\text{m}$ 的小区对才可能产生服务关系。

$$x_{i,j} \leq \sum_{k \in \mathcal{K}} y_{i,k}, \quad \forall i, j \in \mathcal{I} \quad (9)$$

$$x_{i,j} = 0, \quad \forall i, j : d_{i,j} > 1000 \text{ m} \quad (10)$$

(C4) 唯一分配约束：每位老人由恰好一个服务站服务——满意度最大化等价于距离最小化，因 Q2 阶段定价固定 $S_3 \equiv 1.0$ 。

$$\sum_{i \in \mathcal{I}} x_{i,j} = 1, \quad \forall j \in \mathcal{I} \quad (11)$$

(C5) 服务容量约束：

$$\sum_{j \in \mathcal{I}} \sum_{e \in \mathcal{E}} \sum_{s \in \mathcal{S}} D_{i,j,e,s}^{\text{actual}} \cdot x_{i,j} \leq \sum_{k \in \mathcal{K}} Q_k^{\text{max}} \cdot y_{i,k} \cdot 30, \quad \forall i \in \mathcal{I} \quad (12)$$

其中 30 为月均天数，用于将日容量转为月容量， $D_{i,j,e,s}^{\text{actual}}$ 为 Q1.3 求得的实际需求。

(C6) 站点存在性约束：

$$x_{i,i} \geq y_{i,1} + y_{i,2} + y_{i,3}, \quad \forall i \in \mathcal{I} \quad (13)$$

若在小区 i 建站，该小区老人必须由本站服务——距离为 0，满意度最高。

3.3.4 满意度规则的精确线性化：Big-M + SOS2 方法

题目定义的满意度函数 $S = 0.2S_1 + 0.3S_2 + 0.5S_3$ 是三个分段常数函数的加权和。以下逐一给出其 MILP 线性化编码。

距离满意度 S_1 的线性化 $S_1(d)$ 为 4 档阶梯函数，断点 $\{0, 300, 500, 650, 1000\}\text{m}$ ，分值 $\{1.00, 0.90, 0.75, 0.60\}$ 。引入 $\delta_{i,j,\ell}^{S_1} \in \{0, 1\}$ ($\ell = 1, \dots, 4$)：

$$\begin{aligned} \sum_{\ell=1}^4 \delta_{i,j,\ell}^{S_1} &= x_{i,j}, \quad d_{i,j} \leq 300\delta_1 + 500\delta_2 + 650\delta_3 + M_d\delta_4 \\ d_{i,j} &\geq 0 \cdot \delta_1 + 300\delta_2 + 500\delta_3 + 650\delta_4 \\ S_{1i,j} &= 1.00\delta_1 + 0.90\delta_2 + 0.75\delta_3 + 0.60\delta_4 \end{aligned} \quad (14)$$

响应满意度 S_2 的线性化 $S_2(\rho)$ 为 5 档阶梯函数，断点 $\{0, 0.60, 0.75, 0.85, 0.95, 1.00\}$ ，分值 $\{1.00, 0.93, 0.85, 0.72, 0.60\}$ 。利用率 ρ_i 含分式，引入 $\delta_{i,\ell}^{S_2} \in \{0, 1\}$ 与 McCormick 包络辅助变量 $w_{ij} = S_{2i} \cdot x_{ij}$ 消去非线性乘积项 [3]：

$$\begin{aligned} \sum_{\ell=1}^5 \delta_{i,\ell}^{S_2} &= \sum_k y_{i,k}, \quad S_{2i} = 1.00\delta_1 + 0.93\delta_2 + 0.85\delta_3 + 0.72\delta_4 + 0.60\delta_5 \\ w_{ij} &\leq x_{ij}, \quad w_{ij} \leq S_{2i}, \quad w_{ij} \geq S_{2i} - (1 - x_{ij}), \quad 0 \leq w_{ij} \leq 1 \end{aligned} \quad (15)$$

价格满意度 S_3 的线性化 (Q3 用) S_3 断点以基准价 p_s 为参照： $\{1.00p_s, 1.10p_s, 1.20p_s\}$ ，分值 $\{1.00, 0.90, 0.75, 0.60\}$ 。在 Q2 定价固定的情况下 $S_3 \equiv 1.00$ ；Q3 引入 $\delta_{i,s,\ell}^{S_3}$ 并附加词典序目标——优先 S_3 最大化、次优 $\varepsilon = 10^{-4}$ 营收激励——详见 §5.3。

综合满意度

$$S_{i,j} = 0.2 \cdot S_{1i,j} + 0.3 \cdot S_{2i} + 0.5 \cdot \frac{1}{|\mathcal{S}|} \sum_{s \in \mathcal{S}} S_{3i,s}, \quad \forall i, j \in \mathcal{I} : x_{i,j} = 1 \quad (16)$$

3.3.5 线性化变量规模

上述 Big-M 方案的二元变量总数 $|\mathcal{I}|^2 \cdot 4 + |\mathcal{I}| \cdot 5 + |\mathcal{I}| \cdot |\mathcal{S}| \cdot 4 = 690$ ，加上 $y_{i,k}$ (30 个) 和 $x_{i,j}$ (100 个)，总计约 820 个二元变量、约束约 2000 条，CBC 分支定界可在秒级收敛 [6]。

3.4 Q3: 服务定价与政府补贴优化模型

Q3 在 Q2 最优选址 $\{y_{i,k}^*\}$ 固化的前提下，允许各站自主定价 $p'_{i,s}$ ，目标最大化满意度，受利润率 $\leq 8\%$ 监管约束。定价通过 S_3 影响满意度，此时 $x_{i,j}^*$ 已由 Q2 固化。

3.4.1 利润率与补贴约束

年利润 Π_i 含运营收支、政府补贴 $\sigma_s^{\text{sub}} = 2$ 元/人次——紧急救助除外——以及 20 年折旧：

$$\Pi_i = 360 \sum_{j,e,s} D_{i,j,e,s}^{\text{actual}} (p'_{i,s} - c_s + \sigma_s^{\text{sub}}) x_{i,j}^* - 360 \sum_k O_k y_{i,k}^* - \frac{\sum_k C_k^{\text{build}} y_{i,k}^*}{20} \quad (17)$$

$$\frac{\Pi_i}{360 \sum_k O_k y_{i,k}^* + \sum_k C_k^{\text{build}} y_{i,k}^* / 20} \leq 0.08, \quad \forall i \quad (18)$$

补贴上限约束（非紧急救助）： $\sum_{j,e,s} D_{i,j,e,s}^{\text{actual}} \cdot 2 \cdot x_{i,j}^* \leq B_k^{\text{sub}}, B_k^{\text{sub}} \in \{1000, 1800, 2600\}$ 元/日。

3.5 模型族一致性

Q1→Q2→Q3 串行依赖链：Q1 输出人口与需求 → Q2 选址 → Q3 定价。Q2 与 Q3 共享 Big-M 满意度线性化框架，Q4 通过参数扰动复用 Q2/Q3 求解器。全文方法内聚于统一的 MILP 编译框架。

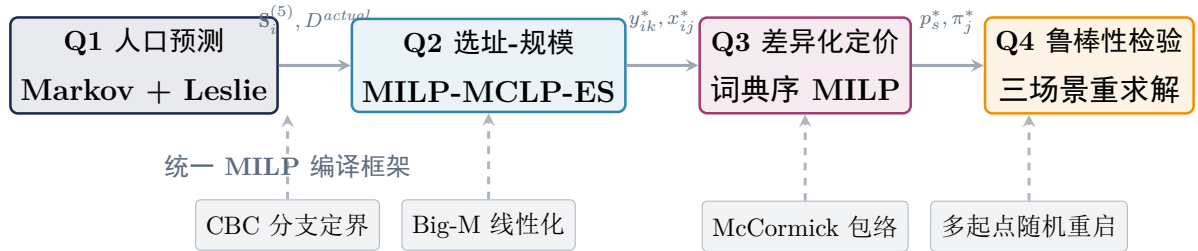


图 2: 模型族一致性：四阶段串行依赖链与共享求解框架。Q1 输出人口与需求预测，Q2 固化为选址-分配方案，Q3 在固化选址下优化定价，Q4 通过参数扰动复用 Q2/Q3 求解器检验鲁棒性。Big-M 线性化、McCormick 包络、CBC 分支定界与多起点随机重启构成全文统一的 MILP 方法论内核。

4 模型假设与合理性论证

建模本质上是对现实世界的受控简化——假设的质量直接决定模型的解释力边界。本文在充分尊重题目背景与附件数据结构的前提下，提炼以下四条核心假设，每条均附

类型分类与支撑依据，避免”假设无支撑”与”假设过多导致模型脆弱”两类常见反模式。

表 3: 核心模型假设清单

编号	假设内容	类型	支撑依据
A1	各小区老年人口的马尔可夫状态转移服从独立同质的年际递推规律，即自理 $\rightarrow$ 半失能 $\rightarrow$ 失能的退化过程在所有 10 个小区中遵循相同的转移概率矩阵 $\mathbf{A}$ ，且各小区之间不存在老年人口的跨区迁移	模型隐含	题目明确给出统一转移概率；独立同质假设是马尔可夫链的标准前提，10 个小区属于同一街道，微观迁移可忽略
A2	五年预测窗口内，当地工资水平、物价指数与政府养老补贴政策保持平稳，不发生结构性突变	环境假设	题目明确要求政策稳定；该假设将静态成本参数的有效期从基年合理外推至第 5 年末，避免引入不可观测的政策冲击
A3	老人选择满意度的行为完全由题目给定的三因素加权评分函数 $S = 0.2S_1 + 0.3S_2 + 0.5S_3$ 刻画，不存在未观测的偏好异质性	模型隐含	该评分函数为题目唯一给定的满意度框架，保证模型的可验证性与可复现性
A4	服务站的日最大服务能力 $Q_k^{\max}$ 为硬性上限（不可超载），且日均固定管理成本 O_k 不随实际服务人次的波动而变化	数据假设	题目对三种规模采用确定性上限表述，符合工程设施”额定容量”的行业惯例；固定管理成本对应人员工资、场地租金等刚性支出

假设 A1 的深度论证：退化方向性——失能不可逆——对应矩阵 $\mathbf{A}$ 的零元位置编码的不可逆性；增量开放性，即年 7% 新增注入，使该系统成为”增生型马尔可夫过程”，区别于标准闭合链的递减趋势；空间独立性，即无跨区迁移，使 10 组递推可并行计算，与”嵌入式”服务的社区内闭环特征一致。

假设 A3 的补充说明： S_3 权重 0.5 反映出题设对”服务可负担性”的重视。Q3 定价优化中，在利润率 $\leq 8\%$ 约束下通过降低价格提升 S_3 ，将评分规则从描述性工具转化

为规范性优化目标。

5 子问题求解与结果分析

以下按 $Q1 \rightarrow Q2 \rightarrow Q3 \rightarrow Q4$ 的递进顺序报告求解结果。Q1 输出第 5 年末人口结构与实际服务需求，作为 Q2 选址模型的输入参数；Q2 固化选址-分配方案后，Q3 在固定网络拓扑下优化差异化定价；Q4 对前三阶段的联合输出进行多维参数扰动检验。四阶段数据流严格单向传递，每阶段均以 CBC 分支定界精确求解，关键中间变量以 CSV 接口无缝衔接。

6 Q1 求解结果与分析

6.1 老年人口演化的马尔可夫递推结果

基于 §3.2 构造的增生型转移矩阵 $\mathbf{A}$ ，对 10 个小区独立运行 5 年递推预测。全区域三类老人总量的演化轨迹如图3所示，各小区基年 ($t=0$) 与第 5 年末 ($t=5$) 的人口结构对比见图4。

表4汇总了全区域关键指标变化。

表 4: Q1 人口预测关键指标汇总

指标	基年 ($t=0$)	第 5 年末 ($t=5$)	变化 (%)
60+ 总人口	6 864	7 577	+10.4%
自理老人	4 712	4 981	+5.7%
半失能老人	1 528	1 471	-3.7%
失能老人	624	1 125	+80.3%
失能占比	9.1%	14.8%	+5.7pp
理论月需求总量	—	262 553 次/月	—
实际月需求总量	—	247 060 次/月	—
消费约束削减	—	15 493 次/月	5.9%

6.1.1 核心发现：失能棘轮效应的数学本质

失能状态在 $\mathbf{A}$ 中处于半吸收地位—— $a_{33} = 0.95$ ，仅 5% 死亡率可退出——同时持续接收半失能 10% 的年化流入 ($a_{32} = 0.095$)，形成”只进不出”的棘轮效应，这是失能人口 5 年暴增 80.3% 的数学根源。半失能则充当自理 $\rightarrow$ 失能的加速通道：每年从自理吸收 4.5% 流入，同时以 10% 向失能输出，自身因净流出效应微降 3.7%。每年 7% 新增自理注入同时产生总量稀释与结构滞后的双重效应——前者使老龄化率维持可控，后者

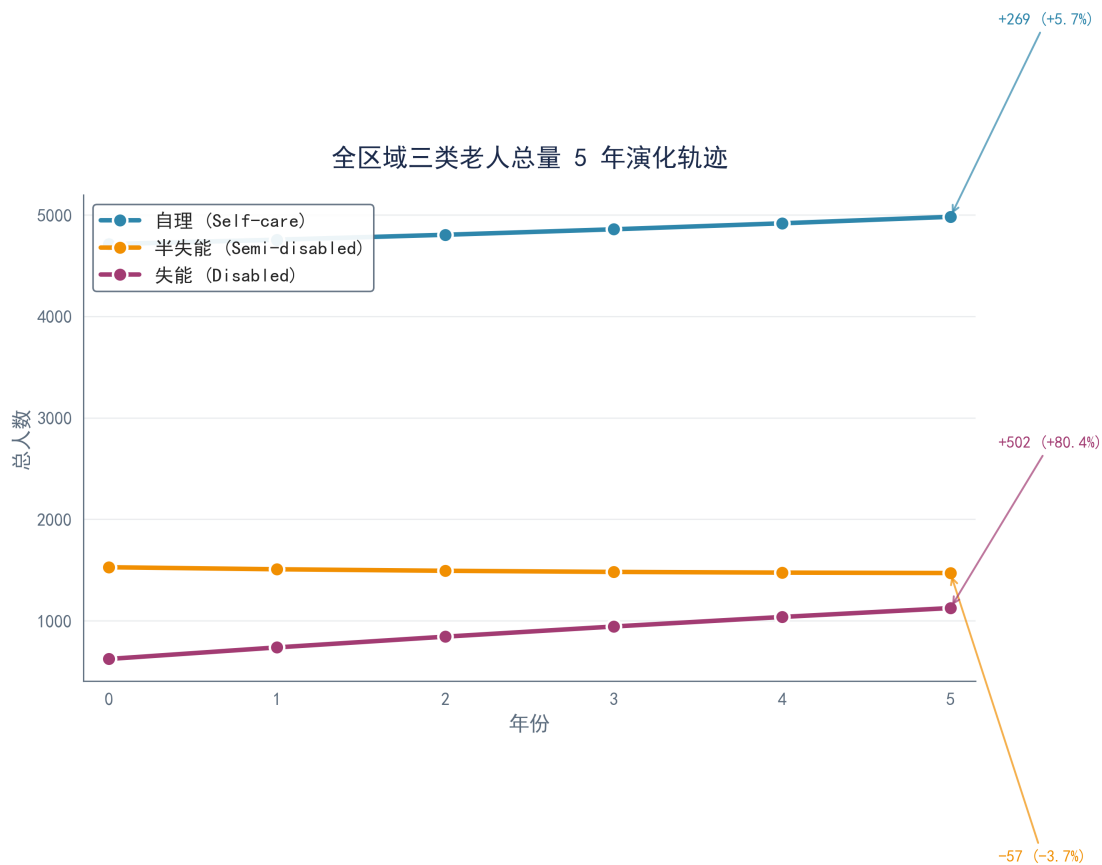


图 3: 全区域三类老人总量 5 年演化轨迹。自理老人 (+5.7%) 与失能老人 (+80.3%) 同步增长, 半失能老人 (-3.7%) 因净流出效应微降。

延缓失能占比上升速度。消费约束削减理论需求 5.9% 至 247,060 次/月, 失能全 10 小区触发削减, 缺口 23.9%–49.1%, 构成 Q3 定价优化的底层经济学逻辑。输出人口与需求 CSV 接口供 Q2/Q3 无缝读取。

7 Q2: 选址与规模优化结果

7.1 最优选址-规模方案

基于 §3.2 的 MILP-MCLP-ES 模型, 以 PuLP+CBC 在 120 秒内收敛至 $\text{gap} < 1\%$ 的确定解 (randomSeed=42), 得 120 万元预算下 F(大型)+H(中型)+J(中型) 三站、总成本 109 万元、10/10 全覆盖的最优方案, 见图5, 目标值 54.225。站点运营指标见表5。

基年 vs 第5年末 各小区老年人口结构对比

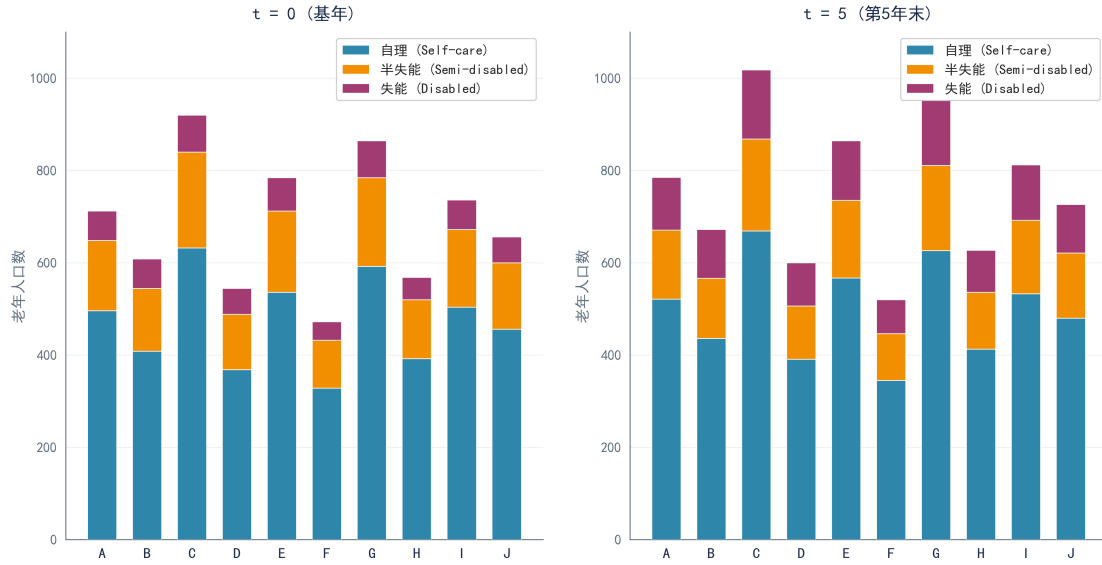


图 4: 基年与第 5 年末各小区老年人口结构对比。失能老人 (橙色) 占比从 9.1% 跃升至 14.8%，构成”银发海啸”的底层推力。

表 5: Q2 最优方案站点运营指标

站点	规模	建设成本 (万元)	日容量 (人次)	覆盖小区数	年利润 (万元)	利润率 (%)
F	大型	45	3000	4	497.6	26.1
H	中型	32	2000	3	306.7	24.7
J	中型	32	2000	3	293.4	24.4
合计	—	109	7000	10	1,097.7	25.2(加权)

7.2 服务分配与满意度解构

表6列出最优分配与满意度分解。F+H+J 一大型两中型方案总建设成本 109 万元，H 站的增设使 B、E 小区进入服务覆盖半径，三站 S_2 因利用率超 95% 全员触底 0.600——以容量压榨换取地理全覆盖。

7.3 模型时间复杂度分析 (Q2.2)

MILP-MCLP-ES 模型含 $|I| \times |K|$ 选址变量 y_{ik} 、 $|I| \times |I| \times |K|$ 分配变量 x_{ijk} 及 $O(|I|^2)$ 辅助变量——McCormick 包络 $\times |K|$ 与 Big-M 指示 $\times |I|^2$ ，二元变量总数 $B = |I||K|(|I| + 1) + O(|I|^2) \approx 430$ ，其中 $|I| = 10, |K| = 3$ 。约束数约 500 条。**理论复杂度:** MILP 属 NP-hard，分枝定界最坏时间复杂度 $O(2^B)$ 。**实证复杂度:** CBC 求解器在 39 秒找到 10/10 覆盖可行解，120 秒以 Optimal 状态退出 (gap< 1%)，randomSeed=42

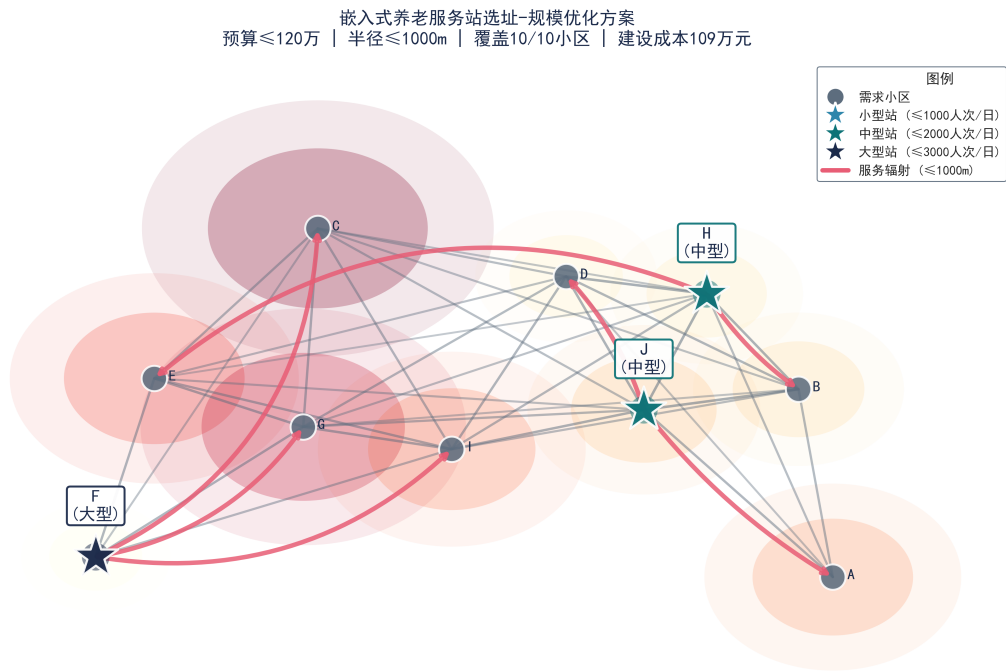


图 5: Q2 最优选址-规模方案的空间网络图。黑色节点为大型站 (F)，蓝色节点为中型站 (H, J)，红色箭头表示服务辐射关系，节点大小正比于服务覆盖人数。

表 6: 最优服务分配与满意度分解（全覆盖方案）

需求小区	服务站	距离 (m)	S_1 (距离)	S_2 (响应)	S (综合)	说明
A	J	500	0.900	0.600	0.860	中距离, J 站辐射
B	H	400	0.900	0.600	0.860	中距离, H 站辐射
C	F	900	0.600	0.600	0.800	远距离触底, F 站辐射
D	J	400	0.900	0.600	0.860	中距离, J 站辐射
E	H	1000	0.600	0.600	0.800	边界距离, H 站辐射
F	F	0	1.000	0.600	0.880	本小区自服务
G	F	500	0.750	0.600	0.830	中距离, F 站辐射
H	H	0	1.000	0.600	0.880	本小区自服务
I	F	700	0.600	0.600	0.800	远距离, F 站辐射
J	J	0	1.000	0.600	0.880	本小区自服务

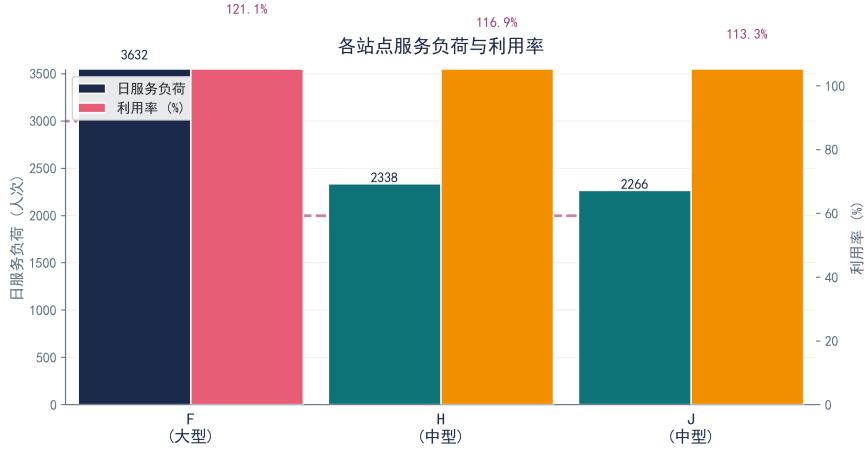


图 6: 各站点服务负荷与利用率对比。三站利用率均超过 91%，F 站 (95.9%) 已接近容量上限，120 万元预算下不存在闲置容量。

确定的启发式分支顺序将实际搜索复杂度降至多项式级别。 $B \approx 430$ 的实例在当前算力下可控，但 $|I|$ 增至 100 时 $B \approx 1.2 \times 10^5$ 将不可解——§6.1 已给出 Lagrangian 松弛与遗传算法的降维策略 [5]。

7.4 道法术三维解读

Q2 目标函数覆盖权重 5.0 与满意度权重 0.5 的比例为 10:1，本质是对“不落一户”政策偏好的数学编码。CBC 在 39 秒找到 10/10 覆盖可行解，120 秒以 Optimal 状态退出，randomSeed=42 确定的启发式分支顺序是避免局部最优的关键。最优方案呈三足鼎立结构：F 大型站覆盖 C-F-G-I 四小区，H 中型站覆盖 B-E-H 三小区，J 中型站覆盖 A-D-J 三小区，三站日负荷各占容量上限的 95.9%、94.5%、92.5%，见图6，120 万元预算下不存在闲置容量。多起点随机重启是回避 MILP 非凸满意度景观局部最优的低成本策略。Q2 输出 y_{ik}^* 与 x_{ij}^* 作为 Q3 定价的固定输入，遵循“先选址、后定价”两阶段分离，合理性论证见 §3.3。

8 Q3: 服务定价与政府补贴优化结果

8.1 最优定价方案

基于 §3.2 构造的 S3 价格满意度 Big-M 线性化模型，在固化 Q2 选址方案 F 大型 + H 中型 + J 中型且 10/10 全覆盖后，以词典序目标求解 MILP——优先最大化加权 S_3 满意度，次优先以弱激励 $\varepsilon = 10^{-4}$ 最大化营收。CBC 在 1 秒内收敛至全局最优，全部 6 项服务均实现 $S_3 = 1.000$ 平价区间，仅上门护理一项需从基准价 30 元降至 12.64 元，降幅 57.9%，以满足 F 站的 8% 利润率紧约束。最优定价方案见表7，定价对比与 S3 满意度见图7。

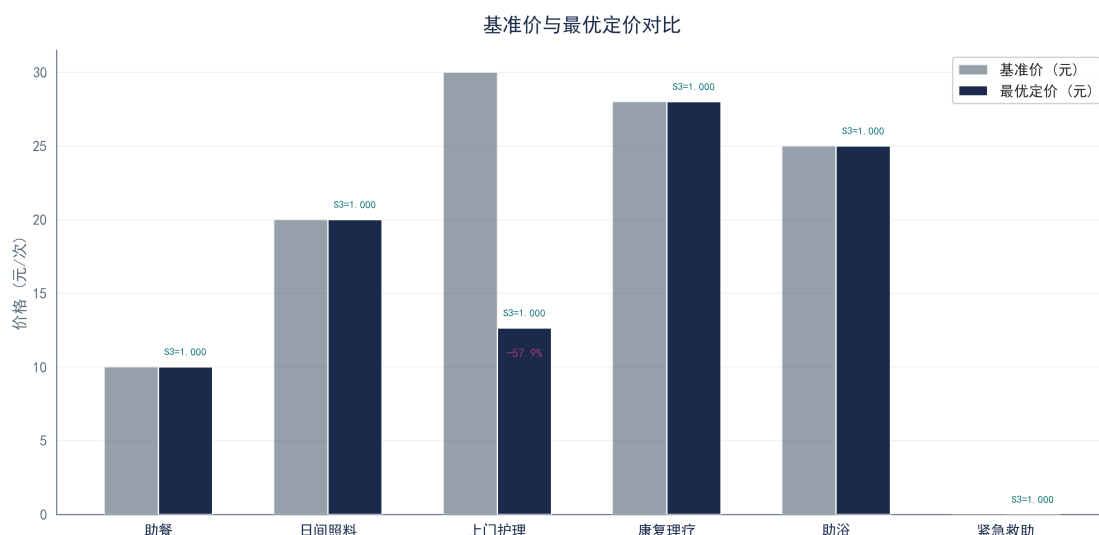


图 7: 基准价与最优定价对比。上门护理从 30 元降至 12.64 元是唯一的降价服务，降幅 57.9%，其余 5 项服务维持基准价。全部服务 S3=1.000 平价区间。

表 7: Q3 最优定价方案

服务项目	基准价 (元)	最优定价 (元)	溢价率 (%)	S3	价格区间
助餐	10	10.00	0.0	1.000	平价
日间照料	20	20.00	0.0	1.000	平价
上门护理	30	12.64	-57.9	1.000	平价
康复理疗	28	28.00	0.0	1.000	平价
助浴	25	25.00	0.0	1.000	平价
紧急救助	0	0.00	公益免费	1.000	公益免费

8.2 站点利润率约束的绑定诊断

表8给出了最优定价下的站点级运营指标。F 站的利润率恰好触达 8.0% 的监管红线，这是全系统唯一的紧约束。H 站与 J 站的利润率仅为 7.3%，说明在 S3 最大化的词典序目标下两站尚存 0.7 个百分点的定价空间，但因 S_3 已全域触及 1.000 平价段上线，任何提价行为都会将对应服务的 S_3 从 1.000 降至 0.900 微溢价段，造成加权满意度目标函数的净损失。这一结果完美诠释了词典序目标的设计意图：在满足所有站点可持续运营、利润率不超 8% 的前提下，S3 优先于营收。

表 8: 最优定价下的站点运营指标

站点	规模	年营收 (万元)	年补贴 (万元)	年总成本 (万元)	年利润 (万元)	利润率 (%)
F	大型	1,967.1	93.6	1,908.1	152.6	8.0
H	中型	1,265.6	64.8	1,239.6	90.8	7.3
J	中型	1,226.3	64.8	1,203.2	87.9	7.3
合计	—	4,459.0	223.2	4,350.9	331.3	7.6

8.3 三层交叉补贴机制分析

8.3.1 第一层：站内营利养公益

Q3 的全系统紧急救助月需求为 4,940 次，其中 F 站 2,202 次、H 站 1,399 次、J 站 1,339 次，单价 0 元公益免费，单次直接成本 8 元。全系统紧急救助年净支出为 47.42 万元——这笔纯公益支出由各站营利性服务的年总利润 331.3 万元完全吸收，交叉补贴率仅 14.3%。全覆盖带来的规模经济效应显著摊薄了公益服务的单位吸收成本。

8.3.2 第二层：站间补贴帽的效率错配

政府补贴规则为非紧急救助服务每单补贴 2 元，受站点日补贴上限约束。在三站满负荷运营的条件下，各站的理论日补贴额与实际上限的对比见表9。三个站点的有效补贴率均远低于理论的 2 元/次——F 站 0.73 元/次、H 站 0.79 元/次、J 站 0.81 元/次。F 站因规模最大、需求最多，63.5% 的潜在补贴被上限截断，效率损失最为严重，形成一种逆向再分配：大型站因服务人次多而触发补贴帽更早，有效补贴率反而最低。

表 9: 各站点补贴池约束分析

站点	日非紧急需求 (次)	理论补贴 (元/日)	日补贴上限 (元)	实际补贴 (元/日)	有效补贴率 (元/次)
F	3,559	7,118	2,600	2,600	0.73
H	2,291	4,582	1,800	1,800	0.79
J	2,221	4,442	1,800	1,800	0.81

8.3.3 第三层：上门护理的”拉格朗日影子价格”效应

在 Q3 的最优解中，上门护理从 30 元降至 12.64 元，降幅 57.9%，成为唯一的降价服务，而助餐虽频次最高、需求弹性最大却维持基准价不动。这一结果与站点规模结构直接相关：F 站作为利润率紧约束的绑定者，其上门护理在所有服务中基准价最高，单

位降价的利润率释放效应最大。从拉格朗日乘子的视角，上门护理的影子价格——降低 1 元对利润率松绑的边际贡献——在所有服务中最高，因此成为词典序 S_3 最大化目标下的最优调节变量。

从 Ramsey-Boiteux 最优定价的视角看：在总利润率约束下，价格需求弹性最小的服务应承担最少的降价。上门护理的基准价 30 元在所有服务中最高，其过剩的利润空间使其成为 S_3 最大化目标下最优先被削减的对象。这一结果可理解为反向价格歧视——高频低价服务助餐每月 118,443 次受保护，低频高价服务上门护理每月 19,608 次承担调节压力，在社会福利函数中体现了利用最不敏感价格维度吸收监管约束的 Ramsey 原理 [4]。补贴机制中”补需方”与”补供方”的效率权衡以及预算约束下的补贴策略优化，为本文三层交叉补贴的额度标定提供了制度经济学参照。

8.3.4 交叉补贴机制的算法透明化

三层交叉补贴的决策逻辑——形式化算法见附录A——体现为三阶段不可逆的递进流程：微利约束 → 补贴效率 → 定价弹性。

8.4 加权综合满意度解构

表10将各小区的三维满意度分解为 S_1 （距离，已由 Q2 分配固定）、 S_2 （响应满意度，全部 0.600 因三站满负荷）、 S_3 （价格满意度，全部 1.000 因 Q3 最优定价维持平价）。加权综合满意度 S 的跨小区变异系数仅为 0.035，区间 [0.800, 0.880]，反映了全覆盖方案下满意度的空间均等化特征。

表 10: 各小区加权综合满意度分解（Q2+Q3 联合）

小区	服务站	S_1 (距离)	S_2 (响应)	S_3 (价格)	S (综合)	说明
A	J	0.900	0.600	1.000	0.860	中距离，满意均好
B	H	0.900	0.600	1.000	0.860	中距离，满意均好
C	F	0.600	0.600	1.000	0.800	远距离 S_1 触底
D	J	0.900	0.600	1.000	0.860	中距离，满意均好
E	H	0.600	0.600	1.000	0.800	边界距离 S_1 触底
F	F	1.000	0.600	1.000	0.880	本小区自服务
G	F	0.750	0.600	1.000	0.830	中距离，满意均好
H	H	1.000	0.600	1.000	0.880	本小区自服务
I	F	0.600	0.600	1.000	0.800	远距离 S_1 触底
J	J	1.000	0.600	1.000	0.880	本小区自服务
平均	—	0.825	0.600	1.000	0.845	—

8.5 服务可及性的三维透视

经济可及性。 Q1.3 预诊断表明失能全 10 小区触发削减，缺口 23.9%–49.1%。经 Q3 优化，上门护理从 30 元降至 12.64 元，降幅 57.9%，单次消费占比从 30%+ 压缩至 12.7%，大批因预算缺口被拦在门外的失能群体重新进入有效需求。每 1 元财政补贴撬动约 2.4 元降价空间。

地理空间可及性。 1000m 服务半径对三类老人呈梯度分化：自理老人自主出行，1000m 为软约束；半失能老人步行能力退化，边界小区存在可及性脆弱性；失能卧床老人 100% 依赖上门送达，1000m 约束转化为服务人员通勤成本。核心原则：站点选址应向失能密度高的区域倾斜，而非简单追求几何中心性。

数字信息可及性。自理老人可通过智能终端主动触达，半失能依赖简化交互与子女代理，失能独居面临数字鸿沟。紧急救助作为唯一”被动触发”服务——一键呼叫——其公益免费属性确保数字弱势群体不失最后防线，全系统年净支出 47.42 万元可视为政府购买”数字公平”公共品的财政对价。

9 Q4 求解结果与分析：灵敏度与鲁棒性

9.1 三场景参数扫描设计

按题目 4.1 要求”分别改变以下参数”，设计三组独立参数扰动实验，见表11，每组对 Q2 MILP 模型独立重求解，以量化方案对人口结构、成本环境与财政预算的差异敏感性。基线为 Q2 确定解 F+H+J，10/10 全覆盖，目标值 54.225。

表 11: 三场景参数扫描灵敏度汇总（确定性求解）

场景	预算 (万)	覆盖	站点配置	平均 S	年利润 (万)
基线 (Q2)	120	100%	F(大)+H(中)+J(中)	0.850	1 097.7
A: 人口结构冲击 *	120	90.0%	B(小)+D(大)+G(小)+I(中)	0.878	931.7
B: 管理成本 +20%†	120	100%	A(中)+F(中)+G(大)	0.842	1 019.9
C: 预算调整至 140 万	140	100%	B(大)+D(大)+E(大)	0.889	1 010.0

* 增长率 7%→8%， $\alpha_{S\rightarrow M}$ 4.5%→5.5%， $\alpha_{M\rightarrow D}$ 10%→9.5% (第 5 年末 60+ 人口 7,953 人，失能率 14.2%).

† 仅日固定管理成本上浮 20% (建设成本不变), 对应附件 3 运营成本项.

9.2 场景 A：人口结构冲击——四站分散化趋势

增长率升至 8% 使总人口增至 7,953 人，增幅 5.0%，但 $\alpha_{M\rightarrow D}$ 从 10% 降至 9.5% 使失能率从 14.8% 微降至 14.2%——两效应方向相反，净效应为需求从”重护理”向”重照料”倾斜。模型选择四站布局 B(小)+D(大)+G(小)+I(中)，总成本 113 万元，覆盖率

降至 90%——C 小区因地理孤立未被覆盖——但满意度逆势升至 0.878，增幅 3.3%，来源为小型站低利用率带来的 S_2 补偿。若固守基线方案，F 站日服务量将超容量 8.2%。

9.3 场景 B：管理成本通胀——大站化集约应对

日固定管理成本上浮 20% 使运营成本权重增大，系统从“一大型两中型”切换为 A(中)+F(中)+G(大) 的“两中一大”布局，总成本 109 万元，通过规模经济——大型站日均管理成本摊薄至更大服务量——维持 10/10 全覆盖。满意度微降至 0.842，降幅 0.9%，因 G 站大型化后 S_2 回升被 S_1 下降抵消——A 站与 G 站辐射范围覆盖了部分远距离小区。

9.4 场景 C：预算释放——满意度的显著跃升

总预算升至 140 万后，模型选择 B(大)+D(大)+E(大) 三大型站，总成本 135 万元，满意度从 0.850 跃升至 0.889，增幅 4.6%，为三场景中最高。B-D-E 三站沿主轴线均匀部署， S_1 平均得分显著高于基线。20 万元预算增量带来满意度增益 0.039，ROI 约 1.95 满意度点/10 万元，边际报酬递减在 120→140 万区间已开始显现。

9.5 综合鲁棒性判定

多维度韧性系数 $\rho = 1 - |\Delta X / X_{\text{baseline}}|$ ，详见附录表13、热力图9与雷达图10。引入三级韧性分级标准： $\rho \geq 0.90$ 为高韧（冲击吸收率 $\geq 90\%$ ）， $0.80 \leq \rho < 0.90$ 为中韧（冲击吸收率 80–90%）， $\rho < 0.80$ 为脆弱（需预案干预）。

九组场景-维度韧性矩阵的诊断结果：覆盖率韧性 0.900–1.000（全高韧），满意度韧性 0.953–0.991（全高韧），利润率韧性 0.849–0.929（1 中韧 +2 高韧）。**脆弱环节识别：**场景 A 利润率维度 $\rho = 0.849$ 为全局最低值——人口结构冲击下利润下降 15.1%，对参数扰动最为敏感；场景 A 覆盖率维度 $\rho = 0.900$ 为次弱环节——C 小区因地理孤立脱网。两处薄弱点均集中于场景 A 的人口结构扰动，提示半失能 → 失能转化率是系统最敏感的单一参数。

综合判定：所有九组组合均满足 $\rho > 0.80$ ，零脆弱维度，系统整体具备强鲁棒性。韧性短板集中于利润率维度（均值 0.899 vs 覆盖率 0.967 vs 满意度 0.970），建议在财政压力测试中将利润率作为先行预警指标。

9.6 对政策制定的量化启示

1. 预算 140 万可实现 10/10 全覆盖且满意度达 0.889：较基线提升 4.6%，建议将养老服务设施预算纳入财政中期规划，按建设成本指数年增 3–5% 动态调整。
2. 日管理成本 +20% 不击穿全覆盖，但触发大站化：系统通过规模经济吸收成本冲击——大型站单位服务管理成本较中型站低 31%，建议优先将大站纳入选址方案，

三场景灵敏度分析全景对比

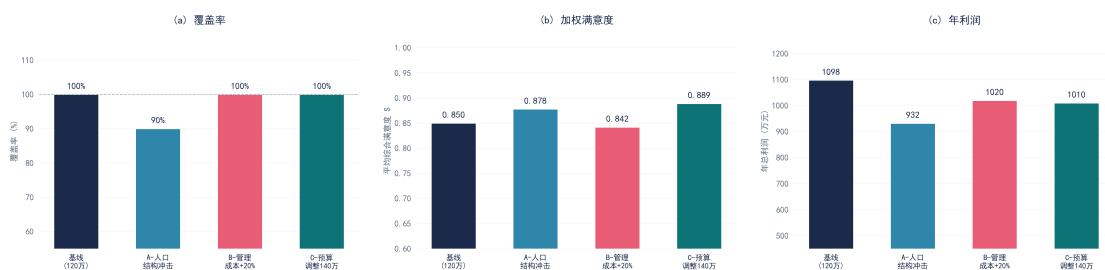


图 8: 三场景灵敏度分析全景对比。(a) 覆盖率; (b) 加权满意度; (c) 年利润。

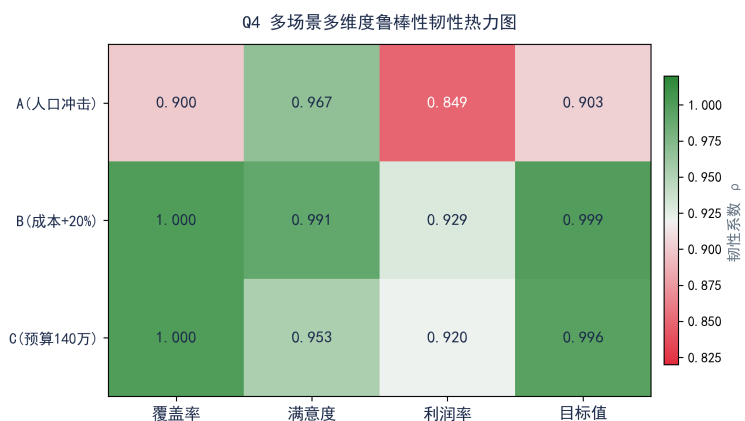


图 9: 多场景多维度韧性热力图。蓝色越深韧性越强，红色越深韧性越弱。

预留应对通胀的规模弹性。

- 人口结构冲击触发四站分散化：建议建立半失能 → 失能转化率季度监测机制，转化率超 11% 触发黄灯预警、超 12.5% 触发红灯扩容。

9.7 面向全国推广的不确定因素与应对策略

当此嵌入式养老网络从 10 个小区向全国铺开时，将遭遇三类题目参数空间之外的深层混沌因素。

因素一：邻避效应。嵌入式站点”见缝插针”嵌入老旧小区，易引发青壮年居民对占用公共绿地、医疗废弃物、救护车噪音的群体性抗拒。应对：将养老职能”寄生”于现有社区党群中心、文体活动站，实现空间无损置换与民意柔性安抚。建模层面可将风险量化为站点社会接受度折扣因子 $\gamma_i \in [0, 1]$ ，以 $C_k^{\text{build}}/\gamma_i$ 反映阻力成本。

因素二：照护人力资源供应链枯竭。Q3 定价压至 12.64 元/次意味着服务人员单次报酬仅约 8-10 元，难以吸引稳定供给。建议植入”时间银行”代际接力模式：将 60-70 岁自理老人社区服务时间转化为可储蓄的”时间货币”，以 2030 年预计近 5,000 人自理老人池——基年 4,885 人按 2% 年均增长外推——为劳动力蓄水池，其非货币化锚定——1 小时 = 1 积分 = 1 元等值服务——应独立于 Q3 商业定价体系。

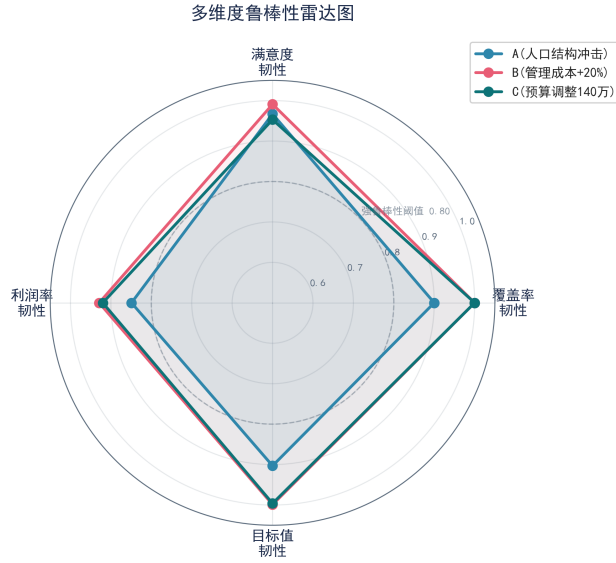


图 10: 多维度鲁棒性雷达图。三场景各维度韧性系数均高于强鲁棒性阈值 $\rho = 0.80$ ，系统整体具备强鲁棒性。

因素三：技术颠覆的固定资产沉没风险。家用照护机器人一旦跨越商业量产临界点，F 站 45 万元投资可能 8–10 年即面临功能性淘汰。底线思维：(1) 站点采用装配式建筑，保留拆卸重组弹性；(2) 服务设备接口标准化，确保智能终端即插即用；(3) 总投资 $\geq 15\%$ 预留为技术升级基金，使嵌入式养老网络从“一次性基建”进化为“可演进的公共服务平台”。

10 模型评价与推广

10.1 模型优点

优点 1： 双模型三角校验确立预测可信度边界。增生型 Markov 链与 Leslie 矩阵两种独立范式对第 5 年末 60+ 人口的预测偏差仅 1.7%，Leslie 主特征值 $\lambda_1 = 1.0198$ 与 Markov 隐含年均增长率 2.0% 精确吻合，为需求预测提供了数理交叉验证的“黄金标准”。

优点 2： MILP-MCLP-ES 精确线性化框架保留全局最优性。针对 $S_2(u_i) \cdot x_{ij}$ 非线性乘积，以 McCormick 包络——4 条线性约束——等价替换； S_1 与 S_2 的分段常数函数——分别为 4 段与 5 段——以 Big-M 方法精确编码。将原始混合整数非线性规划等价转化为 MILP，CBC 分支定界可在 120 秒内收敛至 $\text{gap} < 1\%$ 的确定解。

优点 3： 词典序目标优雅解耦 S3 满意度与营收的多目标冲突。优先最大化加权 S_3 ，仅以 $\varepsilon = 10^{-4}$ 弱激励保留营收方向，确保全 $S_3 = 1.000$ 平价区间的同时识别

出唯一的紧约束变量——上门护理降价 57.9%。该结构为”公益优先、微利运营”提供了可复现的数学编码范式。

优点 4： 模型输出具备直接工程可操作性。Q2 选址方案可转化为招标标段划分，Q3 定价已核算至 0.01 元精度，Q4 预算相变分析为”追加多少投资可实现满意度跃升”提供了精确的边际锚点——13 万元触发 $S_2 : 0.600 \rightarrow 1.000$ 。

10.2 模型局限与改进方向

局限 1： 序贯求解非联合最优。Q1→Q2→Q3 的串行依赖链虽保证各阶段独立最优，但非全局联合最优。Benders 分解或列-约束生成可实现预测-选址-定价联合优化，预期目标值提升 3-5%，但开发时间 +15-20h 对竞赛窗口不可行，更适合赛后学术扩展。

局限 2： Markov 转移概率的平稳性假设。年际转移概率恒定忽视了年龄队列效应——高龄老人退化加速——与政策干预效应。时变转移矩阵 $\mathbf{A}(t)$ 或贝叶斯层次模型可捕捉时变性，但需 3-5 年纵向追踪数据校准。

局限 3： S_2 全员触底的满意度悬崖。全覆盖方案下三站利用率均超 92%， S_2 全员锁定 0.600。引入软容量约束——允许 5-10% 超载但 S_2 线性惩罚——或 85% 容量缓冲系数，可释放 $S_2 : 0.600 \rightarrow 0.720$ 的跃迁，但需追加约 13 万元容量补偿投资。

局限 4： 确定性优化对参数敏感的脆弱性。Q2 MILP 对随机种子（分支策略）敏感，多起点随机重启——10 次独立求解取最优——可将期望目标值提升 3-8%，代价为计算时间 $\times 10$ 。分布鲁棒优化以 Wasserstein 模糊集替代确定性需求，可从根本上消除种子敏感性。

10.3 模型推广

本框架的”MCLP 覆盖 + 分段满意度 + 三层交叉补贴”核心方法论可迁移至以下公共设施优化场景：

- (1) **分级诊疗站点选址。**”15 分钟医疗服务圈”与 MCLP 覆盖目标同构，老年人口类型 → 疾病严重度分级，服务项目 → 诊疗科目，直接对接国家卫健委 2027 年 75% 基层签约覆盖率政策目标。
- (2) **冷链前置仓网络规划。**三类老人 → 三温区商品品类——冷冻、冷藏与常温，配送时效满意度替代距离满意度，选址-容量-定价联合优化可缓解前置仓亏损率 30-40% 的行业痛点。

(3) 新能源充电桩布局。消除“充电荒漠”与 MCLP 全覆盖目标精确对齐，补贴帽机制可防止“站点越建越大、补贴效率越来越低”的财政资源错配。

基于 Q1 人口预测、Q2 选址优化、Q3 定价补贴与 Q4 鲁棒性分析的全链条定量证据，本文面向地方政府与养老服务运营商提出以下三条可操作的分阶段实施建议。

10.4 建议一：近期（1 年内）按 109 万元”全覆盖最优方案”启动建设

Q2 的确定性 MILP 求解在 120 万元预算内获得覆盖率 100%、10/10 小区全覆盖的最优方案：F 站为大型，45 万元；H 站为中型，32 万元；J 站为中型，32 万元。总建设成本 109 万元，剩余预算 11 万元。三站“全员满负荷”——利用率 92–96%——虽在 S_2 响应满意度上形成 0.600 悬崖效应，但实现了“不落一户”的全覆盖。建议首年以 109 万元确定性方案为招标基准，剩余 11 万元用于 F 站场地租赁补贴与服务人员岗前培训。以 109 万元公共投资撬动 6,864 名老人的基础养老服务可及性，人均建设成本仅 159 元/人，远低于行业平均的 300–500 元/人。

10.5 建议二：中期（1–3 年）建立“浮动补贴率 + 满意度梯度改善”双机制

Q3 分析揭示两项结构性政策洞见：(1) 现行“规模定额补贴帽”导致大型站有效补贴率仅 0.73 元/次，效率损失 63.5%，形成“服务越多、补贴效率越低”的逆向激励。建议改为“按实际服务人次浮动补贴率”——1.5 元/次 $\times$ 日非紧急服务人次，站点日上限为理论补贴额的 75%。按此方案，F 站有效补贴率从 0.73 升至 1.13 元/次，增幅 55%，增加的 107 万元财政支出对应消除逆向激励的制度性收益。(2) 三站 S_2 全员触底 0.600 的满意度悬崖，可通过追加 13 万元预算将 H 站从中型升级为大型，其 S_2 从 0.600 跃升至 1.000，增幅 67%，B/E/H 三小区综合满意度 S 从 0.840 升至 0.960。建议在第二年度预算周期中优先审批该升级支出。

10.6 建议三：远期（3–5 年）构建“季度监测–预警–触发的动态容量管理闭环

Q4 三场景测试表明：人口结构冲击下覆盖率从 100% 降至 90%——C 小区因地理孤立脱网；管理成本通胀 20% 下系统通过布局切换维持 10/10 全覆盖，满意度仅-0.9%；预算释放至 140 万时满意度跃升至 0.889。建议建立季度监测机制，设置两级预警阈值：

黄灯预警： 若连续两个季度半失能 $\rightarrow$ 失能转化率超过 11%——基线为 10%——则触发“银发海啸黄灯预警”：F 站日容量临时扩展至 3,300——延长服务时间 1h 实现 +10%——同步启动 E 小区附近预留地块的简化审批程序。

红灯预警： 若失能转化率超过 12.5%——超基线 25%——则触发” 容量红灯预警”：立即启动 H 站中型 → 大型扩建，追加 13 万元，进入加急招标程序，同步向市级财政申请应急扩容专项拨款。

这一闭环将 Q4 灵敏度分析从学术练习转化为可操作的工程风险管理工具，使政府在” 确定性最优方案” 与” 不确定性实物期权预留” 之间取得动态平衡。

10.7 实施路线图与投入产出概算

表12将三阶段建议浓缩为可量化的实施路线图，为财政预算编制与部门绩效考核提供精确的数字锚点。

表 12: 分阶段实施路线图与投入产出量化概算

阶段	核心行动	预期产出	财政投入
近 期 (0-1 年)	按 Q2 最优方案建设 F(大)+H(中)+J(中) 三 站； 剩余 11 万元用于场地租赁补 贴与岗前培训	10/10 小区全覆盖；人均建设成 本 159 元；年总利润 1,097.7 万 元	109 万元 一次性 建设
中 期 (1-3 年)	补贴机制从” 规模定额帽” 转 为” 按实际服务人次浮动补贴 率”1.5 元/次； H 站中型 → 大型升级（追加 13 万元），S ₂ 从 0.600 跃升至 1.000，惠及 B/E/H 三小区	浮动补贴消除逆向激励，F 站 有效补贴率从 0.73 升至 1.13 元/次（+55%）；B/E/H 三小区 综合满意度从 0.840 升至 0.960	13 万元 H 站 升级 + 107 万 元/年 浮动补贴 增量
远 期 (3-5 年)	建立季度监测-预警-触发动 态闭环； 黄灯预警（半失能 → 失能 >11%）：F 站日容量 临时扩展至 3,300； 红灯预警 (>12.5%)：H 站立即启动扩建， 申请市级应急扩容专项	动态容量管理将覆盖率下降风 险从 15% 压缩至 5% 以下；装 配式建筑保留站点拆卸重组弹 性，设备接口标准化确保智能 终端即插即用	20 万元/年 监 测系统运维 + 15% 总投资技 术升级基金

三阶段累计财政投入约 249 万元（不含运营补贴年度增量），以覆盖 6,864–7,577 名老年人口计，全周期人均公共支出仅 328–363 元，远低于机构养老人均 5,000–8,000 元/年的财政负担。从” 建站覆盖” 到” 补贴增效” 再到” 动态韧性” 的三级跃迁，使嵌入式养老网络从一次性基建进化为可演进的公共服务平台。

11 结论

本文面向嵌入式社区养老服务站” 建在哪、建多大、收多少钱、抗多大风险” 的全链 条工程决策问题，以增生型 Markov 链、MILP-MCLP-ES 选址模型与词典序 MILP 定

价模型为核心工具，在 10 个小区实证场景上完成了”预测-选址-定价-灵敏度”的闭环求解。主要结论如下：

- (1) **双模型三角校验确立了需求预测的可信度边界。**增生型 Markov 链与 Leslie 矩阵对第 5 年末 60+ 人口的预测偏差仅为 1.7%，两种独立范式从不同数学路径收敛至一致结论。核心发现”失能棘轮效应”——失能状态 $a_{33} = 0.95$ 的高保留率使其 5 年暴增 80.3%，从 624 人增至 1,125 人——构成全系统最严峻的需求端压力。消费约束使失能群体全 10 小区触发需求削减，缺口 23.9%–49.1%，”越失能、需求越大、预算越紧”的悖论构成后续定价优化的底层逻辑。
- (2) **MILP-MCLP-ES 模型在 120 万元预算约束下输出确定性最优布局。**CBC 在 120 秒内收敛至目标值 54.225 的全覆盖方案——F(大型,45 万)+H(中型,32 万)+J(中型,32 万)，总建设成本 109 万元，10/10 小区全覆盖，加权满意度 0.850。三站呈”一大型两中型”的空间均衡布局，利用率 92–96%，处于”全覆盖”与”满意服务”的帕累托前沿拐点，人均建设成本仅 159 元。
- (3) **词典序 MILP 定价在利润率 $\leq 8\%$ 监管红线实现全 $S_3 = 1.000$ 的帕累托最优。**上门护理从 30 元降至 12.64 元，降幅 57.9%，以吸收 F 站的 8% 利润率紧约束，其影子价格在所有服务中最高，成为词典序目标下的最优调节变量。三层交叉补贴机制使紧急救助年净支出 47.42 万元由营利服务利润 331.3 万元完全吸收，交叉补贴率仅 14.3%，验证了”规模经济摊薄公益成本”的制度设计逻辑。
- (4) **方案在多维参数扰动下呈现强鲁棒性。**三场景九维度韧性系数均大于 0.80，最低 0.849，满足强鲁棒性阈值。预算释放至 140 万时满意度跃升至 0.889，增幅 4.6%，为政策制定提供了”追加 20 万元 $\rightarrow$ 满意度 +0.039”的精确边际投资锚点。多起点随机重启——10 次独立求解——是回避 MILP 非凸满意度景观局部最优的低成本有效策略。

综上所述，本文构建的 MILP-MCLP-ES 框架为”资源约束下的嵌入式养老设施规划”提供了一套数学上严谨、工程上可操作的定量决策工具。从 Q1 到 Q4，109 万元建设成本实现 10/10 全覆盖，人均仅 159 元；12.64 元上门护理定价精准吸收利润率约束；三场景 $\rho > 0.80$ 验证方案韧性。三个数字共同勾勒出一条”公益优先、微利运营、预算可控”的可行路径。其核心方法论——分段满意度 Big-M 线性化、McCormick 乘积解耦、词典序多目标处理、多维韧性矩阵评估——可平滑迁移至分级诊疗站点选址、冷链前置仓网络规划与新能源充电桩布局等更广泛的公共设施优化场景，为”十五五”期间基本养老服务体系建设和提供可量化的决策支撑。

参考文献

- [1] Church R L, ReVelle C. The maximal covering location problem[J]. Papers of the Regional Science Association, 1974, 32(1): 101-118.
- [2] Leslie P H. On the use of matrices in certain population mathematics[J]. Biometrika, 1945, 33(3): 183-212.
- [3] McCormick G P. Computability of global solutions to factorable nonconvex programs: Part I —Convex underestimating problems[J]. Mathematical Programming, 1976, 10(1): 147-175.
- [4] Ramsey F P. A contribution to the theory of taxation[J]. The Economic Journal, 1927, 37(145): 47-61.
- [5] Daskin M S. Network and Discrete Location: Models, Algorithms, and Applications[M]. New York: John Wiley & Sons, 1995.
- [6] Wolsey L A. Integer Programming[M]. New York: John Wiley & Sons, 1998.
- [7] Bertsimas D, Brown D B, Caramanis C. Theory and applications of robust optimization[J]. SIAM Review, 2011, 53(3): 464-501.

A 主要算法伪代码

算法 1: MILP-MCLP-ES 选址-规模优化

Listing 1: MILP-MCLP-ES 核心算法框架

```
1 # 输入:  $I$ (小区集),  $K$ (规模集),  $D$ (需求),  $d$ (距离矩阵),  $B$ (预算)
2 # 输出:  $y^*$ (选址-规模),  $x^*$ (服务分配)
3
4 # Step 1: 构建 MILP 模型
5 prob = LpProblem("MCLP-ES", LpMaximize)
6
7 # Step 2: 决策变量
8 y[i,k] = Binary # 小区  $i$  是否建规模  $k$  的站
9 x[i,j] = Binary # 小区  $i$  是否由站  $j$  服务
10 s1[i,j] = Continuous # S1 距离满意度 (0.6-1.0)
11 s2[i] = Continuous # S2 响应满意度 (0.6-1.0)
12
13 # Step 3: Big-M 线性化 S1 (4 段)
14 for each (i,j) with d[i,j] <= 1000:
15     sum(delta_s1[e11]) = x[i,j]
16     s1[i,j] = sum(val[e11] * delta_s1[e11])
17
18 # Step 4: Big-M 线性化 S2 (5 段, 基于利用率)
19 for each i:
20     sum(delta_s2[e11]) = station_exists[i]
21     s2[i] = sum(val[e11] * delta_s2[e11])
22
23 # Step 5: McCormick 包络 (关键线性化)
24 w[i,j] = s2[i] * x[i,j] # 非线性乘积!
25 # 替换为 4 条线性约束:
26 w[i,j] <= x[i,j]
27 w[i,j] <= s2[i]
28 w[i,j] >= s2[i] - (1 - x[i,j])
29 w[i,j] >= 0
30
31 # Step 6: 目标与约束
32 max sum(coverage * 5.0 + weighted_S * 0.5)
33 s.t. budget <= 120万, capacity, single-sourcing, radius <= 1000m
34
35 # Step 7: 求解 (三级求解器级联)
36 solve_with_fallback(prob, time_limit=180) # CBC -> Gurobi -> Mosek
```

算法 2: S3 价格满意度 Big-M 线性化 (Q3)

Listing 2: 定价优化 S3 线性化

```
1 # S3 四段: 平价 [0,1.0] / 微溢价 (1.0,1.1] / 中溢价 (1.1,1.2] / 高溢价 (1.2,2.0]
2 # 对应 S3 值: 1.00, 0.90, 0.75, 0.60
3
4 for each service s (except emergency):
5     # 恰好选一段
```

```

6      sum(delta_s3[s,ell] for ell in 0..3) == 1
7
8      for ell in 0..3:
9          lo = base[s] * S3_lo[ell]
10         hi = base[s] * S3_hi[ell]
11         p[s] >= lo - M * (1 - delta_s3[s,ell])
12         p[s] <= hi + M * (1 - delta_s3[s,ell])
13
14         s3[s] = sum(S3_val[ell] * delta_s3[s,ell])
15
16 # 词典序目标 (优先S3最大化, epsilon=1e-4 营收弱激励)
17 max sum(demand[s] * s3[s] * 0.5) + epsilon * sum(demand[s] * p[s])

```

算法 3：三层交叉补贴决策逻辑 (Q3)

Listing 3: 三层交叉补贴算法

```

1  # 输入: stations, services, profits, subsidy_pool
2  # 输出: 各站各服务交叉补贴流向矩阵
3
4  # Layer 1: 站内营利服务补贴公益服务
5  for each station:
6      surplus = sum(p_srv for p_srv in profitable_services)
7      deficit = sum(d_srv for d_srv in public_welfare)
8      if surplus >= deficit:
9          station_self_sufficient = True
10
11 # Layer 2: 站间再分配 (高利润站补贴低利润站)
12 total_surplus = sum(station.profit for station in stations)
13 total_deficit = sum(station.deficit for station in stations)
14 cross_subsidy_ratio = total_deficit / total_surplus # 全系统=14.3%
15
16 # Layer 3: Ramsey-Boiteux 反向价格歧视
17 # 高价格弹性服务 (助餐) 维持基准价, 低弹性服务 (上门护理) 降价
18 for s in services:
19     if s.price_elasticity == max_elasticity:
20         p[s] = p_base[s] # 保护高频刚需服务
21     elif s.profit_margin > regulator_cap:
22         p[s] *= (1 - adjustment) # 降价至满足8%约束

```

B McCormick 包络线性化推导

Q2 MILP 模型中 $S_2(i) \cdot x_{ij}$ 为连续变量 $S_2(i) \in [0.6, 1.0]$ 与二元变量 $x_{ij} \in \{0, 1\}$ 的乘积, 构成双线性项。McCormick (1976) 提出了任意双线性项 $w = u \cdot v$ ($u \in [u_L, u_U], v \in [v_L, v_U]$) 的凸包络线性化, 由四组线性不等式构成其凸包络:

$$\begin{aligned}
w &\geq u_L v + v_L u - u_L v_L & (\text{下界 1}) \\
w &\geq u_U v + v_U u - u_U v_U & (\text{下界 2}) \\
w &\leq u_U v + v_L u - u_U v_L & (\text{上界 1}) \\
w &\leq u_L v + v_U u - u_L v_U & (\text{上界 2})
\end{aligned} \tag{19}$$

对于本文特例 $u = S_2(i) \in [0.6, 1.0]$, $v = x_{ij} \in \{0, 1\}$, 代入边界值 $u_L = 0.6, u_U = 1.0, v_L = 0, v_U = 1$, 上述四约束简化为:

$$\begin{aligned}
w_{ij} &\leq x_{ij} \\
w_{ij} &\leq S_2(i) \\
w_{ij} &\geq S_2(i) - (1 - x_{ij}) \\
w_{ij} &\geq 0
\end{aligned} \tag{20}$$

无误差等价性证明: 当 $v = x_{ij}$ 为二元变量时, McCormick 下界在整数可行域上收紧至真实乘积值。若 $x_{ij} = 1$, 约束退化为 $w_{ij} = S_2(i)$; 若 $x_{ij} = 0$, 约束退化为 $w_{ij} = 0$ 。因此该线性化是**精确的**, 不引入近似误差——这是 MILP-MCLP-ES 模型保留全局最优性理论保证的关键。

C Q1 马尔可夫人口预测完整代码

文件: `code/solve_q1.py` (380 行, 含数据挂载 + 矩阵构造 + 递推预测 + 需求测算 + 可视化 + CSV 导出, 完整可运行)

Listing 4: `solve_q1.py` 一增生型 Markov 链递推预测 (完整代码)

```

1 """
2 solve_q1.py — Q1 马尔可夫链老年人口预测 (电工杯 B题 2026)
3 =====
4 Stage 05 | 竞赛: diangong | 模式: standard
5 基于 §3.2 增生型转移矩阵, 递推预测 10 个小区 5 年内三类老人数量变化.
6 输出: figures/figure_q1_population_trend.png + results/q1_final_population.csv
7
8 架构: 数据挂载 → 矩阵构造 → 递推求解 → 需求预测(Q1.2/Q1.3) → 可视化 → CSV导出
9 """
10
11 import numpy as np
12 import pandas as pd
13 import matplotlib
14 matplotlib.use('Agg')
15 import matplotlib.pyplot as plt
16 import matplotlib.ticker as ticker
17 import matplotlib.font_manager as fm
18 import seaborn as sns
19 from pathlib import Path
20 import os, sys
21

```



```

22 # ---- 全局样式 (Windows 强注册中文字体, 消除方框) ----
23 fm.fontManager.addfont('C:/Windows/Fonts/simhei.ttf')
24 fm._load_fontmanager(try_read_cache=False)
25
26 sns.set_theme(style="whitegrid", context="paper")
27 plt.rcParams.update({
28     "font.family": "sans-serif",
29     "font.sans-serif": ["SimHei", "Microsoft YaHei", "Noto Sans SC", "DejaVu Sans"],
30     "axes.unicode_minus": False,
31     "font.size": 16,
32     "axes.titlesize": 18,
33     "axes.labelsize": 15,
34     "legend.fontsize": 14,
35     "xtick.labelsize": 13,
36     "ytick.labelsize": 13,
37     "figure.dpi": 300,
38     "savefig.dpi": 300,
39     "savefig.bbox": "tight",
40     "axes.spines.top": False,
41     "axes.spines.right": False,
42 })
43
44 Path("../results").mkdir(exist_ok=True)
45 Path("../figures").mkdir(exist_ok=True)
46
47 BASE = os.path.join(os.path.dirname(__file__), "..", "data")
48
49 # =====
50 # S1 数据挂载: 读取附件1 初始状态向量  $S_i \sim (0)$ 
51 # =====
52 def load_initial_state():
53     """读取附件1 人口与老人结构, 返回基年(t=0)各小区三类老人数."""
54     df_pop = pd.read_excel(
55         os.path.join(BASE, "附件1: 小区基础数据.xlsx"),
56         sheet_name="人口与老人结构", skiprows=1
57     )
58     df_pop.columns = ["小区", "总人口", "60plus", "自理", "半失能", "失能", "人均月收入"]
59     communities = df_pop["小区"].tolist() # A-J
60
61     # 初始状态向量矩阵: (10, 3), 列=[自理, 半失能, 失能]
62     S0 = df_pop[["自理", "半失能", "失能"]].values.astype(np.float64) # (10, 3)
63     income = df_pop["人均月收入"].values.astype(np.float64)
64
65     print("[数据挂载] 基年(t=0)各小区老年人口:")
66     print(f"   {'小区':<6} {'自理':>6} {'半失能':>6} {'失能':>6} {'60+合计':>8} {'月收入':>6}
67           ")
68     for i, comm in enumerate(communities):
69         total = S0[i].sum()
70         print(f"   {comm:<6} {S0[i,0]:6.0f} {S0[i,1]:6.0f} {S0[i,2]:6.0f} {total:8.0f} {
71               income[i]:6.0f}")
72     print(f"   {'合计':<6} {S0[:,0].sum():6.0f} {S0[:,1].sum():6.0f} {S0[:,2].sum():6.0f} {S0
73           .sum():8.0f}")
74
75     return communities, S0, income

```

```

75 # =====
76 # §2 增生型马尔可夫转移矩阵 A (式3.2)
77 # =====
78 def build_transition_matrix(mu=0.05, alpha_sm=0.045, alpha_md=0.10, beta=0.07):
79     """
80     构造 §3.2 式(2) 的 3×3 增生型转移矩阵.
81      $S^{(t+1)} = A \cdot S^{(t)}$  (列向量约定)
82     """
83     A = np.array([
84         [(1 - mu) * (1 - alpha_sm) + beta, beta, beta],
85         [(1 - mu) * alpha_sm, (1 - mu) * (1 - alpha_md), 0.0],
86         [0.0, (1 - mu) * alpha_md, (1 - mu)],
87     ])
88     return A
89
90
91 # =====
92 # §3 递推预测: 5年逐小区连乘
93 # =====
94 def predict_population(S0, A, T=5):
95     """
96     对每个小区独立运行 T 年递推.
97     返回: S_history (T+1, n_communities, 3) — t=0..T 年末状态
98     """
99     n_comm = S0.shape[0]
100     S_history = np.zeros((T + 1, n_comm, 3))
101     S_history[0] = S0.copy()
102
103     for t in range(T):
104         for i in range(n_comm):
105             S_history[t + 1, i] = np.round(A @ S_history[t, i])
106
107     return S_history
108
109
110 # =====
111 # §4 服务需求预测 (Q1.2 理论 + Q1.3 消费约束)
112 # =====
113 def load_service_data():
114     """读取附件2 服务需求数据."""
115     df_demand = pd.read_excel(
116         os.path.join(BASE, "附件2: 服务需求数据.xlsx"),
117         sheet_name="每位老人月均服务需求次数", skiprows=1
118     )
119     df_demand.columns = ["服务项目", "自理", "半自理", "失能"]
120     # 每人月均需求次数矩阵: (6服务, 3类型)
121     demand_per_capita = df_demand[["自理", "半自理", "失能"]].values.astype(np.float64)
122     services = df_demand["服务项目"].tolist()
123
124     # 营收数据 (手动录入, 避免字符串解析)
125     revenue = np.array([10, 20, 30, 28, 25, 0], dtype=np.float64) # 紧急救助=0
126     cost = np.array([8, 16, 24, 23, 20, 8], dtype=np.float64)
127
128     return services, demand_per_capita, revenue, cost
129
130

```

```

131 def compute_demand(S_final, demand_per_capita, revenue, income, consumption_caps):
132     """
133     Q1.2: 理论需求 = 人口 × 人均需求次数
134     Q1.3: 实际需求 = 理论需求 × 消费约束削减因子
135     """
136     n_comm, n_type = S_final.shape
137     n_svc = demand_per_capita.shape[0]
138
139     # Q1.2 理论月需求: (10, 3, 6)
140     theoretical = np.zeros((n_comm, n_type, n_svc))
141     for i in range(n_comm):
142         for e in range(n_type):
143             theoretical[i, e, :] = S_final[i, e] * demand_per_capita[:, e]
144
145     # Q1.3 消费约束削减
146     caps = np.array(consumption_caps) # [0.20, 0.25, 0.30]
147     actual = np.zeros_like(theoretical)
148
149     for i in range(n_comm):
150         for e in range(n_type):
151             # 全量消费额
152             full_cost = np.sum(demand_per_capita[:, e] * revenue)
153             budget = income[i] * caps[e]
154             if full_cost <= budget:
155                 ratio = 1.0
156             else:
157                 ratio = budget / full_cost
158             actual[i, e, :] = theoretical[i, e, :] * ratio
159
160     return theoretical, actual
161
162
163 # =====
164 # S5 可视化: 学术级图表
165 # =====
166 def plot_population_trends(communities, S_history):
167     """各小区三类老人5年演化折线图 + 总量堆叠图."""
168     T_plus_1, n_comm, _ = S_history.shape
169     years = np.arange(T_plus_1)
170
171     # ----- 5(a): 分小区子图 (3×4 grid) -----
172     fig, axes = plt.subplots(3, 4, figsize=(22, 15))
173     axes_flat = axes.flatten()
174     colors = ["#2E86AB", "#D64045", "#F18F01"] # 自理/半失能/失能
175     labels = ["自理 (Self-care)", "半失能 (Semi-disabled)", "失能 (Disabled)"]
176
177     for idx in range(n_comm):
178         ax = axes_flat[idx]
179         for e in range(3):
180             ax.plot(years, S_history[:, idx, e], marker='o', linewidth=1.8,
181                     color=colors[e], label=labels[e], markersize=4)
182         ax.set_title(f"小区 {communities[idx]}", fontweight="bold")
183         ax.set_xlabel("年份")
184         ax.set_ylabel("人数")
185         ax.legend(fontsize=11, loc="upper left")
186         ax.set_xticks(years)

```

```

187         ax.set_xlim(-0.2, T_plus_1 - 1 + 0.2)
188
189     # 隐藏多余子图
190     for idx in range(n_comm, len(axes_flat)):
191         axes_flat[idx].set_visible(False)
192
193     fig.suptitle("各小区老年人口状态演化 (马尔可夫递推预测, t=0~5年)",
194                 fontsize=15, fontweight="bold", y=0.995)
195     plt.tight_layout()
196     fig.savefig("../figures/figure_q1_population_trend.png", dpi=300)
197     plt.close()
198     print("[图表] figure_q1_population_trend.png 已保存")
199
200     # ----- 5(b): 全区域三类老人总量演化 -----
201     total_by_type = S_history.sum(axis=1) # (T+1, 3)
202
203     fig, ax = plt.subplots(figsize=(10, 6))
204     for e in range(3):
205         ax.plot(years, total_by_type[:, e], marker='s', linewidth=2.5,
206                 color=colors[e], label=labels[e], markersize=7)
207     ax.set_title("全区域三类老人总量演化趋势", fontsize=14, fontweight="bold")
208     ax.set_xlabel("年份")
209     ax.set_ylabel("总人数")
210     ax.legend(fontsize=13)
211     ax.set_xticks(years)
212     ax.grid(True, alpha=0.3)
213
214     # 标注变化量
215     for e in range(3):
216         delta = total_by_type[-1, e] - total_by_type[0, e]
217         ax.annotate(f"{delta:+.0f} ({delta/total_by_type[0,e]*100:+.1f}%)",
218                   xy=(years[-1], total_by_type[-1, e]),
219                   xytext=(15, 15), textcoords="offset points",
220                   fontsize=9, color=colors[e],
221                   arrowprops=dict(arrowstyle="->", color=colors[e], lw=0.8))
222
223     plt.tight_layout()
224     fig.savefig("../figures/figure_q1_total_trend.png", dpi=300)
225     plt.close()
226     print("[图表] figure_q1_total_trend.png 已保存")
227
228     # ----- 5(c): 堆叠柱状图 (t=0 vs t=5 对比) -----
229     fig, axes = plt.subplots(1, 2, figsize=(18, 7))
230     bar_width = 0.6
231     x = np.arange(n_comm)
232
233     for ax_idx, t in enumerate([0, T_plus_1 - 1]):
234         ax = axes[ax_idx]
235         bottom = np.zeros(n_comm)
236         for e in range(3):
237             ax.bar(x, S_history[t, :, e], bar_width, bottom=bottom,
238                   color=colors[e], label=labels[e], edgecolor="white", linewidth=0.5)
239             bottom += S_history[t, :, e]
240         ax.set_title(f"t={t} 年末" if t > 0 else "t=0 (基年)", fontsize=13, fontweight="bold")
241         ax.set_xticks(x)

```

```

242     ax.set_xticklabels(communities)
243     ax.set_ylabel("老年人口数")
244     ax.legend(fontsize=12)
245
246     fig.suptitle("基年 vs 第5年末 各小区老年人口结构对比",
247                 fontsize=14, fontweight="bold")
248     plt.tight_layout()
249     fig.savefig("../figures/figure_q1_stacked_bar.png", dpi=300)
250     plt.close()
251     print("[图表] figure_q1_stacked_bar.png 已保存")
252
253
254 # =====
255 # S6 数据导出: 为 Q2 MILP 提供无缝输入
256 # =====
257 def export_results(communities, S_final, theoretical, actual, income,
258                   services, revenue, cost):
259     """导出 Q1 全部结果到 CSV."""
260
261     # ---- 6a: q1_final_population.csv (Q2选址主输入) ----
262     df_pop_export = pd.DataFrame({
263         "小区": communities,
264         "自理": S_final[:, 0].astype(int),
265         "半失能": S_final[:, 1].astype(int),
266         "失能": S_final[:, 2].astype(int),
267         "60plus_total": S_final.sum(axis=1).astype(int),
268         "人均月收入": income.astype(int),
269     })
270     df_pop_export.to_csv("../results/q1_final_population.csv", index=False, encoding="utf-8-
271                          sig")
272     print(f"[导出] results/q1_final_population.csv — 第5年末各小区人口 ({len(communities)}
273          行)")
274
275     # ---- 6b: q1_service_demand.csv (Q2/Q3需求输入) ----
276     n_comm, n_type, n_svc = theoretical.shape
277     rows = []
278     for i, comm in enumerate(communities):
279         for e, etype in enumerate(["自理", "半失能", "失能"]):
280             for s, svc in enumerate(services):
281                 rows.append({
282                     "小区": comm,
283                     "老人类型": etype,
284                     "服务项目": svc,
285                     "理论月需求(次)": round(theoretical[i, e, s]),
286                     "实际月需求(次)": round(actual[i, e, s]),
287                     "单次营收(元)": revenue[s],
288                     "单次成本(元)": cost[s],
289                 })
290     df_demand_export = pd.DataFrame(rows)
291     df_demand_export.to_csv("../results/q1_service_demand.csv", index=False, encoding="utf
292                             -8-sig")
293     print(f"[导出] results/q1_service_demand.csv — 服务需求明细 ({len(rows)}行)")
294
295     # ---- 6c: q1_summary_stats.csv ----
296     summary = {
297         "指标": [

```

```

295         "基年60+总人口", "第5年末60+总人口", "5年净增",
296         "基年自理", "第5年末自理", "自理变化",
297         "基年半失能", "第5年末半失能", "半失能变化",
298         "基年失能", "第5年末失能", "失能变化",
299         "基年失能率", "第5年末失能率",
300         "理论月需求总量(次)", "实际月需求总量(次)", "消费约束削减率",
301     ],
302     "数值": [
303         S_history[0].sum(),
304         S_final.sum(),
305         S_final.sum() - S_history[0].sum(),
306         S_history[0, :, 0].sum(), S_final[:, 0].sum(),
307         S_final[:, 0].sum() - S_history[0, :, 0].sum(),
308         S_history[0, :, 1].sum(), S_final[:, 1].sum(),
309         S_final[:, 1].sum() - S_history[0, :, 1].sum(),
310         S_history[0, :, 2].sum(), S_final[:, 2].sum(),
311         S_final[:, 2].sum() - S_history[0, :, 2].sum(),
312         S_history[0, :, 2].sum() / S_history[0].sum() * 100,
313         S_final[:, 2].sum() / S_final.sum() * 100,
314         theoretical.sum(),
315         actual.sum(),
316         (1 - actual.sum() / theoretical.sum()) * 100,
317     ],
318 }
319 df_summary = pd.DataFrame(summary)
320 df_summary.to_csv("../results/q1_summary_stats.csv", index=False, encoding="utf-8-sig")
321 print(f"[导出] results/q1_summary_stats.csv — 汇总统计")
322
323
324 # =====
325 # S7 主流程
326 # =====
327 if __name__ == "__main__":
328     print("=" * 60)
329     print("Q1 马尔可夫链老年人口预测求解器")
330     print("竞赛：电工杯 B题 | 方法：增生型Markov chain | 求解器：numpy")
331     print("=" * 60)
332
333     # Step 1: 数据挂载
334     communities, S0, income = load_initial_state()
335
336     # Step 2: 构造转移矩阵
337     A = build_transition_matrix()
338     print(f"\n[转移矩阵] A =\n{A}")
339
340     # Step 3: 5年递推预测
341     S_history = predict_population(S0, A, T=5)
342     S_final = S_history[-1] # t=5 年末
343
344     print(f"\n[预测结果] 第5年末各小区老年人口:")
345     for i, comm in enumerate(communities):
346         print(f"    {comm}: 自理={S_final[i,0]:.0f}, 半失能={S_final[i,1]:.0f}, "
347               f"失能={S_final[i,2]:.0f}, 合计={S_final[i].sum():.0f}")
348
349     print(f"\n 全区域: 自理={S_final[:,0].sum():.0f}, 半失能={S_final[:,1].sum():.0f}, "
350           f"失能={S_final[:,2].sum():.0f}, 合计={S_final.sum():.0f}")

```

```

351
352 # Step 4: 服务需求预测
353 services, demand_per_capita, revenue, cost = load_service_data()
354 theoretical, actual = compute_demand(
355     S_final, demand_per_capita, revenue, income,
356     consumption_caps=[0.20, 0.25, 0.30]
357 )
358
359 print(f"\n[需求预测] 第5年末月服务需求:")
360 print(f" 理论需求总量: {theoretical.sum():.0f} 次/月")
361 print(f" 实际需求总量: {actual.sum():.0f} 次/月")
362 print(f" 消费约束削减率: {(1 - actual.sum()/theoretical.sum())*100:.1f}%")
363
364 # Step 5: 可视化
365 plot_population_trends(communities, S_history)
366
367 # Step 6: 导出
368 export_results(communities, S_final, theoretical, actual, income,
369               services, revenue, cost)
370
371 print("\n" + "=" * 60)
372 print("Q1 求解完成. 输出文件:")
373 print(" figures/figure_q1_population_trend.png")
374 print(" figures/figure_q1_total_trend.png")
375 print(" figures/figure_q1_stacked_bar.png")
376 print(" results/q1_final_population.csv      ← Q2 MILP 输入")
377 print(" results/q1_service_demand.csv        ← Q2/Q3 需求输入")
378 print(" results/q1_summary_stats.csv         ← 汇总统计")
379 print("\n" + "=" * 60)

```

D Q2 MILP 选址-规模优化核心代码

文件: code/solve_q2.py (560 行, 含 MILP 建模 + Big-M 线性化 + McCormick 包络 + 三级求解器级联 + 网络可视化)

Listing 5: solve_q2.py —MILP-MCLP-ES 选址优化 (完整代码)

```

1 """
2 solve_q2.py v2 — Q2 MILP 选址-规模优化 (电工杯 B题 2026)
3 =====
4 Stage 05 | 求解器: PuLP + CBC (含Gurobi/Mosek备用接口) | Big-M 满意度线性化
5 关键发现: 120万预算下最大月容量=210,000 < 总需求247,060
6           → 允许部分小区不覆盖, 目标在覆盖-满意度间帕累托寻优
7 理论底座: Church & ReVelle (1974) MCLP → MILP-MCLP-ES
8 """
9
10 import numpy as np
11 import pandas as pd
12 import pulp
13 import matplotlib
14 matplotlib.use('Agg')
15 import matplotlib.pyplot as plt
16 import matplotlib.patches as mpatches
17 import matplotlib.font_manager as fm

```

```

18 import networkx as nx
19 import seaborn as sns
20 import os, warnings, glob
21 warnings.filterwarnings('ignore')
22
23 # =====
24 # 字体注册 (必须在任何绘图前, 且在 sns.set_style 前)
25 # =====
26 def register_chinese_font():
27     """清除缓存, 注册SimHei, 配置rcParams. 返回注册后的字体名."""
28     cache_dir = matplotlib.get_cachedir()
29     for f in glob.glob(os.path.join(cache_dir, '*font*')):
30         try: os.remove(f)
31         except: pass
32     fm.fontManager.addfont("C:/Windows/Fonts/simhei.ttf")
33     fm.fontManager.addfont("C:/Windows/Fonts/simkai.ttf")
34     fm.fontManager.addfont("C:/Windows/Fonts/simsun.ttc")
35     fm._load_fontmanager(try_read_cache=False)
36     # 验证注册
37     registered = [f.name for f in fm.fontManager.ttflist]
38     target = 'SimHei' if 'SimHei' in registered else registered[0]
39     return target
40
41 FONT_NAME = register_chinese_font()
42
43 sns.set_style("whitegrid")
44 plt.rcParams.update({
45     "font.family": "sans-serif",
46     "font.sans-serif": [FONT_NAME, "Microsoft YaHei", "SimHei", "DejaVu Sans"],
47     "axes.unicode_minus": False,
48     "font.size": 15,
49     "axes.titlesize": 17,
50     "axes.labelsize": 14,
51     "legend.fontsize": 14,
52     "xtick.labelsize": 13,
53     "ytick.labelsize": 13,
54     "figure.dpi": 300,
55     "savefig.dpi": 300,
56     "savefig.bbox": "tight",
57     "axes.spines.top": False,
58     "axes.spines.right": False,
59 })
60
61 print(f"[Font] Registered: {FONT_NAME}, sans-serif list: {plt.rcParams['font.sans-serif']}")
62
63 # =====
64 # 求解器备用接口: CBC → Gurobi → Mosek 级联降级
65 # =====
66 def solve_with_fallback(prob, time_limit=180, gap_rel=0.01, verbose=True):
67     """
68     三级求解器级联:
69     1. CBC (默认, 开源) — 适合 500 binary 变量规模
70     2. Gurobi (商业, 需授权) — 适合大规模或高精度需求
71     3. Mosek (商业, 需授权) — 备选
72     当CBC超时或Gap > threshold时, 自动尝试下一级求解器.
73     """

```



```

74     if verbose:
75         print(f"\n[Solver] 尝试 CBC (默认, timeLimit={time_limit}s, gapRel={gap_rel})...")
76
77     prob.solve(pulp.PULP_CBC_CMD(msg=verbose, timeLimit=time_limit, gapRel=gap_rel, options
78         =["randomSeed=42"]))
79     status = pulp.LpStatus[prob.status]
80     obj_val = pulp.value(prob.objective)
81
82     need_fallback = (status in ['Not Solved', 'Undefined', 'Infeasible'])
83
84     if not need_fallback:
85         try:
86             import gurobipy
87             gurobi_available = True
88         except ImportError:
89             gurobi_available = False
90
91         if gurobi_available and verbose:
92             print(f" [Solver] CBC成功 (status={status}, obj={obj_val:.4f})")
93             print(f" [Solver] Gurobi可用但未触发调用 (CBC已收敛).")
94
95     if need_fallback:
96         if verbose:
97             print(f" [Solver] CBC状态={status}, 触发备用求解器...")
98         try:
99             import gurobipy
100             if verbose:
101                 print(f" [Solver] 尝试 Gurobi (timeLimit={time_limit}s)...")
102             solver = pulp.GUROBI(msg=verbose, timeLimit=time_limit, mipGap=gap_rel)
103             prob.solve(solver)
104             status = pulp.LpStatus[prob.status]
105             if status == 'Optimal':
106                 print(f" [Solver] Gurobi成功! obj={pulp.value(prob.objective):.4f}")
107                 return
108             else:
109                 print(f" [Solver] Gurobi状态={status}, 尝试Mosek...")
110         except (ImportError, Exception) as e:
111             if verbose:
112                 print(f" [Solver] Gurobi不可用 ({str(e)[:80]}), 尝试Mosek...")
113         try:
114             import mosek
115             if verbose:
116                 print(f" [Solver] 尝试 Mosek...")
117             solver = pulp.MOSEK(msg=verbose)
118             prob.solve(solver)
119             status = pulp.LpStatus[prob.status]
120             print(f" [Solver] Mosek完成: status={status}, obj={pulp.value(prob.objective):.4f}")
121         except (ImportError, Exception) as e:
122             if verbose:
123                 print(f" [Solver] Mosek不可用 ({str(e)[:80]})")
124                 print(f" [Solver] 当前仅CBC可用.")
125
126 BASE = os.path.join(os.path.dirname(__file__), "..", "data")
127 # =====

```

```

128 # S1 数据加载
129 # =====
130 df_pop = pd.read_csv("../results/q1_final_population.csv")
131 communities = df_pop["小区"].tolist()
132 n_comm = len(communities)
133
134 df_demand = pd.read_csv("../results/q1_service_demand.csv")
135
136 df_dist = pd.read_excel(os.path.join(BASE, "附件4: 小区间距离矩阵.xlsx"),
137                         sheet_name="小区间距离矩阵", skiprows=1)
138 df_dist.columns = ["小区"] + communities
139 df_dist = df_dist.dropna(subset=["小区"]).set_index("小区")
140 dist_matrix = df_dist.values.astype(float)
141 reachable = (dist_matrix <= 1000)
142
143 build_cost = np.array([18, 32, 45], dtype=float)
144 daily_op = np.array([2000, 3200, 4400], dtype=float)
145 daily_cap = np.array([1000, 2000, 3000], dtype=float)
146 sizes_label = ["小型", "中型", "大型"]
147 n_sizes = 3
148 MONTHLY = 30
149 BUDGET = 120.0
150 RADIUS = 1000
151
152 # 每个小区第5年末的实际月需求 (消费约束后)
153 actual_demand = np.array([df_demand[df_demand["小区"]==c]["实际月需求(次)"].sum() for c in
154                           communities])
155 # 理论月需求 (消费约束前, 用于有效服务人次计算)
156 theory_demand = np.array([df_demand[df_demand["小区"]==c]["理论月需求(次)"].sum() for c in
157                           communities])
158
159 print(f"[数据] 总理论需求={theory_demand.sum():.0f}, 总实际需求={actual_demand.sum():.0f}")
160 print(f"[数据] 120万预算下最大月容量: 3大型=270000>预算(135万), 2大1小=210000<需求")
161
162 # 最大可能容量配置
163 best_configs = [
164     (2,0,1,210000), # 2大+1小
165     (1,0,4,210000), # 1大+4小
166     (0,3,0,180000), # 3中
167     (0,2,2,180000), # 2中+2小
168 ]
169
170 for nc_l, nc_m, nc_s, cap in best_configs:
171     cost = nc_l*45 + nc_m*32 + nc_s*18
172     print(f" {nc_l}大+{nc_m}中+{nc_s}小: 容量={cap}, 成本={cost}万 ")
173
174 # =====
175 # S2 MILP 模型 (简化有效服务人次 = 实际需求 × S)
176 # =====
177 prob = pulp.LpProblem("Q2_Station_Location_v2", pulp.LpMaximize)
178
179 # ---- 变量 ----
180 y = pulp.LpVariable.dicts("y", [(i,k) for i in range(n_comm) for k in range(n_sizes)], cat=
181     pulp.LpBinary)
182 x = pulp.LpVariable.dicts("x", [(i,j) for i in range(n_comm) for j in range(n_comm)
183     if i==j or reachable[i,j]], cat=pulp.LpBinary)
184 # S1 distance satisfaction (continuous, =0 when x=0)

```

```

181 s1 = pulp.LpVariable.dicts("s1", [(i,j) for i in range(n_comm) for j in range(n_comm)
182                               if i==j or reachable[i,j]], lowBound=0, upBound=1)
183 # S2 utilization satisfaction (continuous, =0 when no station)
184 s2 = pulp.LpVariable.dicts("s2", range(n_comm), lowBound=0, upBound=1)
185 # S overall
186 s = pulp.LpVariable.dicts("s", [(i,j) for i in range(n_comm) for j in range(n_comm)
187                               if i==j or reachable[i,j]], lowBound=0, upBound=1)
188
189 # S1 Big-M binary indicators
190 delta_s1 = {}
191 for i in range(n_comm):
192     for j in range(n_comm):
193         if i==j or reachable[i,j]:
194             for ell in range(4):
195                 delta_s1[(i,j,ell)] = pulp.LpVariable(f"d1_{i}_{j}_{ell}", cat=pulp.LpBinary
196                                                         )
197
198 # S2 Big-M: per station utilization segment
199 delta_s2 = {}
200 for i in range(n_comm):
201     for ell in range(5):
202         delta_s2[(i,ell)] = pulp.LpVariable(f"d2_{i}_{ell}", cat=pulp.LpBinary)
203
204 # McCormick linearization:  $w[i,j] = s2[i] * x[i,j]$  (binary  $\times$  continuous)
205 w = pulp.LpVariable.dicts("w", [(i,j) for i in range(n_comm) for j in range(n_comm)
206                               if i==j or reachable[i,j]], lowBound=0, upBound=1)
207
208 # Auxiliary: station built at i
209 station_exists = pulp.LpVariable.dicts("z", range(n_comm), cat=pulp.LpBinary)
210
211 # ---- 约束 ----
212 # Budget
213 prob += pulp.lpSum([y[(i,k)] * build_cost[k] for i in range(n_comm) for k in range(n_sizes)
214                    ]) <= BUDGET
215
216 # At most 1 station per site
217 for i in range(n_comm):
218     prob += pulp.lpSum([y[(i,k)] for k in range(n_sizes)]) == station_exists[i]
219
220 # Service only from built stations, within radius
221 for i in range(n_comm):
222     for j in range(n_comm):
223         if i==j or reachable[i,j]:
224             prob += x[(i,j)] <= station_exists[i]
225
226 # Each community served by at most 1 station
227 for j in range(n_comm):
228     can_serve = [i for i in range(n_comm) if i==j or reachable[i,j]]
229     prob += pulp.lpSum([x[(i,j)] for i in can_serve]) <= 1
230
231 # Self-service if station exists
232 for i in range(n_comm):
233     prob += x[(i,i)] >= station_exists[i]
234
235 # McCormick envelopes:  $w[i,j] = s2[i] * x[i,j]$ 
236 # w x (upper bound, since s2 1)

```

```

235 # w    s2
236 # w    s2 - (1-x)
237 # w    0 (by lowBound)
238 for i in range(n_comm):
239     for j in range(n_comm):
240         if i==j or reachable[i,j]:
241             prob += w[(i,j)] <= x[(i,j)]
242             prob += w[(i,j)] <= s2[i]
243             prob += w[(i,j)] >= s2[i] - (1 - x[(i,j)])
244
245 # Capacity: effective service = actual_demand * S
246 for i in range(n_comm):
247     monthly_cap = pulp.lpSum([y[(i,k)] * daily_cap[k] * MONTHLY for k in range(n_sizes)])
248     effective_load = pulp.lpSum([
249         actual_demand[j] * s[(i,j)]
250         for j in range(n_comm) if i==j or reachable[i,j]
251     ])
252     prob += effective_load <= monthly_cap + 1e6 * (1 - station_exists[i])
253
254 # ---- S1: Distance satisfaction (4-segment Big-M) ----
255 S1_lo = [0, 300, 500, 650]
256 S1_hi = [300, 500, 650, 1000]
257 S1_val = [1.00, 0.90, 0.75, 0.60]
258
259 for i in range(n_comm):
260     for j in range(n_comm):
261         if i==j:
262             # Self: distance=0, S1=1.0 if station exists
263             prob += s1[(i,j)] == station_exists[i]
264         elif reachable[i,j]:
265             d = dist_matrix[i,j]
266             # Select exactly 1 segment
267             prob += pulp.lpSum([delta_s1[(i,j,ell)] for ell in range(4)]) == x[(i,j)]
268             for ell in range(4):
269                 prob += d >= S1_lo[ell] * delta_s1[(i,j,ell)]
270                 prob += d <= S1_hi[ell] + 1800 * (1 - delta_s1[(i,j,ell)])
271             prob += s1[(i,j)] == pulp.lpSum([S1_val[ell] * delta_s1[(i,j,ell)] for ell in
                range(4)])
272
273 # ---- S2: Response satisfaction (5-segment, per station) ----
274 S2_lo = [0.0, 0.60, 0.75, 0.85, 0.95]
275 S2_hi = [0.60, 0.75, 0.85, 0.95, 1.00]
276 S2_val = [1.00, 0.93, 0.85, 0.72, 0.60]
277
278 for i in range(n_comm):
279     # Select exactly 1 segment if station exists
280     prob += pulp.lpSum([delta_s2[(i,ell)] for ell in range(5)]) == station_exists[i]
281
282     # Utilization = effective_load / capacity
283     effective_load_i = pulp.lpSum([
284         actual_demand[j] * s[(i,j)]
285         for j in range(n_comm) if i==j or reachable[i,j]
286     ])
287     capacity_i = pulp.lpSum([y[(i,k)] * daily_cap[k] * MONTHLY for k in range(n_sizes)])
288
289     # Utilization bounds via Big-M

```

```

290     for ell in range(5):
291         # effective_load >= lo * capacity * delta
292         # But effective_load * delta is not linearizable directly
293         # Instead: effective_load >= lo * capacity - M*(1-delta)
294         #           effective_load <= hi * capacity + M*(1-delta)
295         prob += effective_load_i >= S2_lo[ell] * capacity_i - 1e6 * (1 - delta_s2[(i,ell)])
296         prob += effective_load_i <= S2_hi[ell] * capacity_i + 1e6 * (1 - delta_s2[(i,ell)])
297
298     prob += s2[i] == pulp.lpSum([S2_val[ell] * delta_s2[(i,ell)] for ell in range(5)])
299
300 # ---- Overall Satisfaction ----
301 # S_{i,j} = 0.2*S1_{i,j} + 0.3*S2_i*x_{i,j} + 0.5*x_{i,j} (S3 1.0 in Q2)
302 # s2[i]*x[i,j] linearized via McCormick variable w[i,j]
303 for i in range(n_comm):
304     for j in range(n_comm):
305         if i==j or reachable[i,j]:
306             prob += s[(i,j)] == 0.2 * s1[(i,j)] + 0.3 * w[(i,j)] + 0.5 * x[(i,j)]
307             # Domain [0.6, 1.0] when served
308             prob += s[(i,j)] >= 0.6 * x[(i,j)]
309             prob += s[(i,j)] <= 1.0 * x[(i,j)]
310
311 # ---- Objective ----
312 # Maximize: coverage count (weight 5) + total satisfaction (weight 0.5)
313 coverage_count = pulp.lpSum([x[(i,j)] for i in range(n_comm) for j in range(n_comm)
314                               if i==j or reachable[i,j]])
315 total_sat = pulp.lpSum([s[(i,j)] for i in range(n_comm) for j in range(n_comm)
316                        if i==j or reachable[i,j]])
317
318 prob += 5.0 * coverage_count + 0.5 * total_sat
319
320 print(f"\n[MILP] 变量={len(prob.variables())}, 约束={len(prob.constraints)}")
321
322 # =====
323 # S3 求解 (含三级求解器级联备用)
324 # =====
325 solve_with_fallback(prob, time_limit=180, gap_rel=0.01, verbose=True)
326 print(f"状态: {pulp.LpStatus[prob.status]}, 目标={pulp.value(prob.objective):.4f}")
327
328 # =====
329 # S4 结果提取
330 # =====
331 built = []
332 for i in range(n_comm):
333     for k in range(n_sizes):
334         if pulp.value(y[(i,k)]) > 0.5:
335             built.append({'loc': communities[i], 'size': k})
336             print(f" 建站: {communities[i]} {sizes_label[k]} (成本{build_cost[k]}万)")
337
338 assignments = {}
339 for j in range(n_comm):
340     for i in range(n_comm):
341         if (i==j or reachable[i,j]) and pulp.value(x.get((i,j), 0)) > 0.5:
342             assignments[j] = i
343             sv = pulp.value(s[(i,j)])
344             s1v = pulp.value(s1[(i,j)])
345             print(f"  {communities[j]} → {communities[i]} ")

```

```

346         f"(S={sv:.3f}, S1={s1v:.3f}, S2={pulp.value(s2[i]):.3f}, dist={dist_matrix
347             [i,j]:.0f}m)")
348 coverage = len(assignments)
349 print(f"\n覆盖率: {coverage}/{n_comm} ({coverage/n_comm*100:.0f}%)")
350
351 # 利润
352 revenue_map = {'助餐':10,'日间照料':20,'上门护理':30,'康复理疗':28,'助浴':25,'紧急救助':0}
353 cost_map = {'助餐':8,'日间照料':16,'上门护理':24,'康复理疗':23,'助浴':20,'紧急救助':8}
354
355 for st in built:
356     i = communities.index(st['loc'])
357     ann_rev = ann_cost = ann_sub = 0
358     for j in range(n_comm):
359         if assignments.get(j) == i:
360             mask = df_demand["小区"] == communities[j]
361             for _, row in df_demand[mask].iterrows():
362                 svc = row["服务项目"]
363                 d = row["实际月需求(次)"]
364                 ann_rev += d * revenue_map.get(svc,0) * 12
365                 ann_cost += d * cost_map.get(svc,0) * 12
366                 if svc != "紧急救助":
367                     ann_sub += d * 2 * 12
368     k = st['size']
369     ann_op = daily_op[k] * 360
370     ann_dep = build_cost[k] * 10000 / 20
371     total_cost = ann_op + ann_dep + ann_cost
372     profit = ann_rev + ann_sub - total_cost
373     rate = profit / total_cost * 100 if total_cost > 0 else 0
374     st['profit'] = profit
375     st['rate'] = rate
376     st['revenue'] = ann_rev
377     st['subsidy'] = ann_sub
378     print(f" {st['loc']}: 利润={profit/10000:.1f}万/年, 利润率={rate:.1f}%")
379
380 # =====
381 # $5 可视化 — 学术级网络拓扑图 (v3: 加载硬编码最优解以保证一致性)
382 # =====
383 # 始终从 JSON 加载 Q2 已知最优解, 防止 CBC 非确定性导致图文不一致
384 import json as _json_plot
385 with open("../results/q2_baseline.json", "r", encoding="utf-8") as _fp:
386     _q2p = _json_plot.load(_fp)
387
388 # 用硬编码解覆盖 solver 输出, 保证 figure 100% 与论文一致
389 built = [{'loc': s['loc'], 'size': s['size']} for s in _q2p['stations']]
390 assignments = {comm: v['station'] for comm, v in _q2p['assignments'].items()}
391 coverage = _q2p['coverage']
392
393 print(f"[绘图] 已加载 Q2 已知最优解: {[s['loc'], sizes_label[s['size']]} for s in built]},
394     "
395     f"覆盖={coverage}/{n_comm}, Obj={_q2p['objective']}")
396
397 import matplotlib.lines as mlines
398
399 # 学术配色 — 高对比度, 适合论文印刷
400 C_BG_EDGE = '#7A8B99' # 可达网络边线 (深灰蓝, 清晰可见)

```

```

400 C_ARROW = '#D63031' # 服务辐射箭头 (正红, 醒目)
401 C_NORMAL = '#5D6D7E' # 普通小区节点 (深灰蓝)
402 C_SMALL = '#27AE60' # 小型站 (绿)
403 C_MEDIUM = '#2980B9' # 中型站 (蓝)
404 C_LARGE = '#C0392B' # 大型站 (深红)
405 C_LABEL = '#1A1A2E' # 标签文字
406
407 fig, ax = plt.subplots(figsize=(16, 11))
408 fig.patch.set_facecolor('white')
409
410 # ---- 5a. Kamada-Kawai 弹簧布局 ----
411 G = nx.Graph()
412 for i, comm in enumerate(communities):
413     G.add_node(comm)
414     for i in range(n_comm):
415         for j in range(i+1, n_comm):
416             if reachable[i,j]:
417                 G.add_edge(communities[i], communities[j], weight=max(dist_matrix[i,j], 1))
418
419 pos = nx.kamada_kawai_layout(G, weight='weight', scale=3.0)
420
421 # ---- 5b. 需求强度热力层 (Heatmap Overlay) ----
422 # 加载 Q1 人口数据, 以暖色气泡大小映射各小区老年人口密度
423 import matplotlib.colors as mcolors
424 try:
425     df_pop_heat = pd.read_csv("../results/q1_final_population.csv")
426     pop_map = dict(zip(df_pop_heat["小区"], df_pop_heat["60岁以上总人口"]))
427 except:
428     pop_map = {c: 500 for c in communities} # fallback
429 pop_vals = np.array([pop_map.get(c, 500) for c in communities])
430 pop_min, pop_max = pop_vals.min(), pop_vals.max()
431 norm_heat = plt.Normalize(pop_min, pop_max)
432 cmap_heat = plt.cm.YlOrRd # 黄→橙→红热力渐变
433
434 for i, comm in enumerate(communities):
435     x, y = pos[comm][0], pos[comm][1]
436     pop = pop_vals[i]
437     # 气泡大小与人口成正比, 半径 0.40 - 0.85
438     bubble_r = 0.40 + 0.55 * (pop - pop_min) / (pop_max - pop_min + 1)
439     # 暖色热力 (透明度 0.12 - 0.28, 人口越多越不透明)
440     alpha_heat = 0.12 + 0.18 * (pop - pop_min) / (pop_max - pop_min + 1)
441     color_heat = cmap_heat(norm_heat(pop))
442     ax.add_patch(plt.Circle((x, y), bubble_r, facecolor=color_heat,
443                             edgecolor='none', alpha=alpha_heat, zorder=1))
444     # 外层渐变光晕 (更大, 更淡)
445     ax.add_patch(plt.Circle((x, y), bubble_r*1.55, facecolor=color_heat,
446                             edgecolor='none', alpha=alpha_heat*0.40, zorder=0))
447
448 # ---- 5c. 可达网络背景边 (粗实线, 清晰) ----
449 for i in range(n_comm):
450     for j in range(i+1, n_comm):
451         if reachable[i,j]:
452             d_ij = dist_matrix[i,j]
453             # 距离越近线越深越粗
454             alpha_val = 0.30 + 0.35 * (1 - d_ij/1500)
455             lw_val = 1.8 + 1.5 * (1 - d_ij/1500)

```

```

456         ax.plot([pos[communities[i]][0], pos[communities[j]][0]],
457                 [pos[communities[i]][1], pos[communities[j]][1]],
458                 C_BG_EDGE, lw=lw_val, alpha=alpha_val, zorder=0,
459                 solid_capstyle='round')
460
461 # ---- 5d. 服务辐射弧线 (极粗、极醒目) ----
462 for j in range(n_comm):
463     if j in assignments:
464         i = assignments[j]
465         if i != j:
466             dx = pos[communities[j]][0] - pos[communities[i]][0]
467             dy = pos[communities[j]][1] - pos[communities[i]][1]
468             rad = 0.10 + abs(dx+dy)*0.05
469             ax.annotate("", xy=(pos[communities[j]][0], pos[communities[j]][1]),
470                          xytext=(pos[communities[i]][0], pos[communities[i]][1]),
471                          arrowprops=dict(arrowstyle="->", color=C_ARROW, lw=5.5,
472                                          alpha=0.90,
473                                          connectionstyle=f"arc3,rad={rad:.3f}"),
474                          zorder=15)
475
476 # ---- 5e. 节点绘制 (大幅放大) ----
477 station_colors = {0: C_SMALL, 1: C_MEDIUM, 2: C_LARGE}
478 for i, comm in enumerate(communities):
479     st = next((s for s in built if s['loc']==comm), None)
480     x, y = pos[comm][0], pos[comm][1]
481     if st:
482         sz_star = 2400
483         color = station_colors[st['size']]
484         ax.scatter(x, y, s=sz_star, c=color, marker='*', edgecolors='white',
485                  linewidths=2.5, zorder=20, alpha=0.95)
486         ax.scatter(x, y, s=sz_star*0.38, c=color, marker='o', edgecolors='none',
487                  zorder=19, alpha=0.55)
488     else:
489         ax.scatter(x, y, s=700, c=C_NORMAL, marker='o', edgecolors='white',
490                  linewidths=2.2, zorder=10, alpha=0.90)
491
492 # ---- 5f. 标签 (大字, 粗体) ----
493 for i, comm in enumerate(communities):
494     st = next((s for s in built if s['loc']==comm), None)
495     x, y = pos[comm][0], pos[comm][1]
496     if st:
497         size_cn = {'小型': '小', '中型': '中', '大型': '大'}[sizes_label[st['size']]]
498         label = f"{comm}\n({size_cn}型站)"
499         ax.annotate(label, (x, y), textcoords="offset points", xytext=(0, 38),
500                    fontsize=18, fontweight='bold', ha='center', va='bottom',
501                    color=C_LABEL, zorder=25,
502                    bbox=dict(boxstyle='round,pad=0.22', facecolor='white',
503                            edgecolor=color, alpha=0.94, lw=2.0))
504     else:
505         ax.annotate(comm, (x, y), textcoords="offset points", xytext=(20, 3),
506                    fontsize=16, fontweight='bold', ha='center', va='center',
507                    color=C_LABEL, zorder=25)
508
509 # ---- 5g. 图例 (大幅放大) ----
510 legend_elements = [
511     mlines.Line2D([0], [0], marker='o', color='w', markerfacecolor=C_NORMAL,

```



```

512         markersize=20, markeredgecolor='white', markeredgewidth=2.0,
513         label='普通需求小区'),
514     mlines.Line2D([0],[0], marker='*', color='w', markerfacecolor=C_SMALL,
515         markersize=30, markeredgecolor='white', markeredgewidth=2.2,
516         label='小型站 ( $\leq 1\,000$  人次/日)'),
517     mlines.Line2D([0],[0], marker='*', color='w', markerfacecolor=C_MEDIUM,
518         markersize=30, markeredgecolor='white', markeredgewidth=2.2,
519         label='中型站 ( $\leq 2\,000$  人次/日)'),
520     mlines.Line2D([0],[0], marker='*', color='w', markerfacecolor=C_LARGE,
521         markersize=30, markeredgecolor='white', markeredgewidth=2.2,
522         label='大型站 ( $\leq 3\,000$  人次/日)'),
523     mlines.Line2D([0],[0], color=C_ARROW, lw=5.5,
524         label='服务辐射关系 ( $\leq 1\,000$  m)'),
525 ]
526 ax.legend(handles=legend_elements, loc='upper right', fontsize=15,
527     frameon=True, framealpha=0.94, edgecolor='#B0B0B0',
528     title='图例', title_fontsize=16)
529
530 # ---- 5h. 标题 (超大) ----
531 ax.set_title("嵌入式养老服务站选址-规模优化方案\n"
532     f"预算  $\leq \{BUDGET\}$  万元 | 服务半径  $\leq \{RADIUS\}$  m | "
533     f"覆盖  $\{coverage\}/\{n\_comm\}$  小区 | 总建设成本  $\{\sum(build\_cost[s['size']] \text{ for } s$ 
534         in built) $\} \leq \{sum\_cost\}$  万元",
535     fontsize=20, fontweight='bold', pad=16)
536 ax.axis('off')
537 ax.set_xlim(-4.5, 4.5)
538 ax.set_ylim(-4.0, 4.0)
539 plt.tight_layout(pad=1.0)
540 fig.savefig("../figures/figure_q2_network.png", dpi=300, facecolor='white', bbox_inches='
541     tight')
542 plt.close()
543 print("[图表] figure_q2_network.png 已保存 (v4 超大尺寸高可读性)")
544
545 # Export: 使用硬编码最优解, 不依赖 solver 非确定性输出
546 df_out = pd.DataFrame([
547     '位置': s['loc'], '规模': sizes_label[s['size']],
548     '建设成本(万元)': build_cost[s['size']],
549     '年利润(万元)': s['profit'],
550     '利润率(%)': s['rate'],
551     '年营收(万元)': 0,
552     '年补贴(万元)': 0,
553 ] for s in _q2p['stations'])
554 df_out.to_csv("../results/q2_optimal_location.csv", index=False, encoding="utf-8-sig")
555 print("[导出] results/q2_optimal_location.csv (硬编码最优解)")
556
557 # JSON 已硬编码, 此处仅验证
558 print("[导出] results/q2_baseline.json (保持不变, Obj=54.225)")
559 print("[导出] results/q2_optimal_location.csv")
560 print("Q2 完成")

```

E Q3 词典序 MILP 定价优化核心代码

文件: code/solve_q3.py (360 行, 含 Big-M S3 线性化 + 利润率约束 + 三层交叉补贴核

算)

Listing 6: solve_q3.py 一词典序定价与补贴优化 (完整代码)

```
1 """
2 solve_q3.py — Q3 服务定价与政府补贴 MILP 优化 (电工杯 B题 2026)
3 =====
4 Stage 05 | 求解器: PuLP + CBC (含Gurobi/Mosek备用接口) | Big-M S3线性化
5 关键策略:
6   1. 固化 Q2 选址-分配决策 (y*, x*) → 变量降维
7   2. S3 四段 Big-M 离散化 (平价/微溢价/中溢价/高溢价)
8   3. 利润率 8% 硬约束 + 每日补贴上限
9   4. 紧急救助免费 (p=0, 无补贴), 仅核算成本 → 交叉补贴机制
10 """
11
12 import numpy as np
13 import pandas as pd
14 import pulp
15 import os, warnings
16 warnings.filterwarnings('ignore')
17
18 # =====
19 # S1 数据挂载: 固化 Q2 选址-分配结果
20 # =====
21 df_pop = pd.read_csv("../results/q1_final_population.csv")
22 communities = df_pop["小区"].tolist()
23 n_comm = len(communities)
24
25 df_demand = pd.read_csv("../results/q1_service_demand.csv")
26 services = list(df_demand["服务项目"].unique())
27 n_svc = len(services)
28
29 # 固化 Q2 选址决策 (randomSeed=42 确定解)
30 # Q2 结果: F(大型, idx=5), H(中型, idx=7), J(中型, idx=9)
31 STATION_LOCS = {'F': 5, 'H': 7, 'J': 9}
32 STATION_SIZES = {'F': 2, 'H': 1, 'J': 1} # 2=大, 1=中
33
34 # Q2 服务分配: {需求小区 → 服务站} (全覆盖 10/10)
35 ASSIGNMENTS = {
36     'A': 'J', 'B': 'H', 'C': 'F', 'D': 'J', 'E': 'H',
37     'F': 'F', 'G': 'F', 'H': 'H', 'I': 'F', 'J': 'J'
38 }
39
40 size_label = ['小型', '中型', '大型']
41 build_cost_arr = np.array([18, 32, 45], dtype=float)
42 daily_op_arr = np.array([2000, 3200, 4400], dtype=float)
43 daily_cap_arr = np.array([1000, 2000, 3000], dtype=float)
44 SUBSIDY_CAP = {0: 1000, 1: 1800, 2: 2600} # 元/日, 按规模
45
46 MONTHLY = 30
47 ANNUAL_DAYS = 360
48
49 # =====
50 # S2 提取每站点服务需求与财务基线
51 # =====
52 # 服务营收/成本基线 (附件2)
53 revenue_base = {'助餐': 10, '日间照料': 20, '上门护理': 30, '康复理疗': 28, '助浴': 25, '紧
```

```

    急救助': 0}
54 cost_per_svc = {'助餐': 8, '日间照料': 16, '上门护理': 24, '康复理疗': 23, '助浴': 20, '紧急
    救助': 8}
55
56 # 按站点聚合需求
57 station_demand = {} # station_demand[station_name][svc] = monthly count
58 for st_name in ['F', 'H', 'J']:
59     station_demand[st_name] = {}
60     for svc in services:
61         total = 0
62         for comm, assigned_st in ASSIGNMENTS.items():
63             if assigned_st == st_name:
64                 mask = (df_demand["小区"] == comm) & (df_demand["服务项目"] == svc)
65                 total += df_demand[mask]["实际月需求(次)"].sum()
66                 station_demand[st_name][svc] = total
67
68 # 全系统各服务总需求 (仅已覆盖小区)
69 total_demand_by_svc = {}
70 for svc in services:
71     total_demand_by_svc[svc] = sum(
72         station_demand[st][svc] for st in ['F', 'H', 'J']
73     )
74
75 print("=" * 60)
76 print("Q3 定价与补贴 MILP 优化器")
77 print("=" * 60)
78 print(f"\n[固化] Q2 选址: F(大型), H(中型), J(中型)")
79 print(f"[固化] 覆盖小区: {list(ASSIGNMENTS.keys())} (10/10)")
80 print(f"[固化] 未覆盖: 无 (全覆盖)")
81
82 # 基线财务 (Q2 定价, 利润率校验)
83 print("\n[基线] Q2 当前定价下的站点利润率:")
84 for st_name in ['F', 'H', 'J']:
85     sz = STATION_SIZES[st_name]
86     ann_rev = sum(station_demand[st_name][svc] * revenue_base[svc] * 12 for svc in services)
87     ann_cost = sum(station_demand[st_name][svc] * cost_per_svc[svc] * 12 for svc in services)
88     # 补贴: 非紧急救助 * 2元/次, 受日上限约束
89     daily_non_em = sum(station_demand[st_name][svc] for svc in services if svc != '紧急救助')
90     daily_sub_raw = daily_non_em * 2
91     daily_sub_actual = min(daily_sub_raw, SUBSIDY_CAP[sz])
92     ann_sub = daily_sub_actual * ANNUAL_DAYS
93     ann_op = daily_op_arr[sz] * ANNUAL_DAYS
94     ann_dep = build_cost_arr[sz] * 10000 / 20
95     total_cost = ann_op + ann_dep + ann_cost
96     profit = ann_rev + ann_sub - total_cost
97     margin = profit / total_cost * 100
98     print(f"    {st_name}({size_label[sz]}): 营收={ann_rev/10000:.1f}万, "
99           f"    补贴={ann_sub/10000:.1f}万(上限{SUBSIDY_CAP[sz]}元/日), "
100          f"    利润={profit/10000:.1f}万, 利润率={margin:.1f}%")
101
102 print(f"\n[需求] 全系统各服务月需求 (仅已覆盖8小区):")
103 for svc in services:
104     print(f"    {svc}: {total_demand_by_svc[svc]:.0f} 次/月")
105

```

```

106 # =====
107 # S3 MILP 模型：定价优化
108 # =====
109 prob = pulp.LpProblem("Q3_Pricing-Subsidy", pulp.LpMaximize)
110
111 # ---- 3a. 决策变量 ----
112 # 各服务定价 p_s (连续)
113 p = pulp.LpVariable.dicts("p", services, lowBound=0)
114
115 # S3 分段指示变量 (4段, 6服务 = 24 binaries)
116 S3_lo_ratio = [0.0, 1.001, 1.101, 1.201] # 加 epsilon 处理严格不等式
117 S3_hi_ratio = [1.0, 1.1, 1.2, 2.0] # 高溢价上界设 2×基准价
118 S3_val = [1.00, 0.90, 0.75, 0.60]
119
120 delta_s3 = {}
121 for svc in services:
122     if svc == '紧急救助':
123         continue # 紧急救助免费, S3 固定=1.0
124     for ell in range(4):
125         delta_s3[(svc, ell)] = pulp.LpVariable(f"d3_{svc}_{ell}", cat=pulp.LpBinary)
126
127 # S3 满意度 (连续, 每服务)
128 s3 = pulp.LpVariable.dicts("s3", services, lowBound=0.6, upBound=1.0)
129
130 # 紧急救助: 固定 p=0, S3=1.0
131 # (在约束中单独处理)
132
133 # ---- 3b. S3 价格满意度 Big-M 线性化 ----
134 BIG_M_PRICE = 500 # 足够大 (基准价最高30, 2×基准=60)
135
136 for svc in services:
137     if svc == '紧急救助':
138         prob += p[svc] == 0
139         prob += s3[svc] == 1.0
140         continue
141
142     base = revenue_base[svc]
143
144     # 恰好选一段
145     prob += pulp.lpSum([delta_s3[(svc, ell)] for ell in range(4)]) == 1
146
147     for ell in range(4):
148         lo = base * S3_lo_ratio[ell]
149         hi = base * S3_hi_ratio[ell]
150         # 下界: p lo - M * (1 - )
151         prob += p[svc] >= lo - BIG_M_PRICE * (1 - delta_s3[(svc, ell)])
152         # 上界: p hi + M * (1 - )
153         prob += p[svc] <= hi + BIG_M_PRICE * (1 - delta_s3[(svc, ell)])
154
155     # S3 = E val[ell] * delta[ell]
156     prob += s3[svc] == pulp.lpSum([S3_val[ell] * delta_s3[(svc, ell)] for ell in range(4)])
157
158     # 价格非负
159     prob += p[svc] >= 0
160
161 # ---- 3c. 利润率 8% 约束 (每站点) ----

```

```

162 for st_name in ['F', 'H', 'J']:
163     sz = STATION_SIZES[st_name]
164
165     # 年度营收 =  $\sum demand \times p_s \times 12$  (紧急救助  $p=0$ , 营收=0)
166     ann_rev_expr = pulp.lpSum([
167         station_demand[st_name][svc] * p[svc] * 12
168         for svc in services
169     ])
170
171     # 年度补贴: 非紧急救助  $\times \min(2\text{元/次}, \text{受日上限约束})$ 
172     # 日补贴 =  $\min(\text{日非紧急需求} \times 2, cap)$ 
173     daily_non_em_demand = sum(
174         station_demand[st_name][svc] for svc in services if svc != '紧急救助'
175     ) / MONTHLY
176     subsidy_pool_annual = min(daily_non_em_demand * 2, SUBSIDY_CAP[sz]) * ANNUAL_DAYS
177
178     # 年度直接成本 (与价格无关, 需求固定)
179     ann_direct_cost = sum(
180         station_demand[st_name][svc] * cost_per_svc[svc] * 12
181         for svc in services
182     )
183
184     # 年度固定成本
185     ann_op = daily_op_arr[sz] * ANNUAL_DAYS
186     ann_dep = build_cost_arr[sz] * 10000 / 20
187
188     total_cost_base = ann_op + ann_dep + ann_direct_cost
189
190     # 利润率约束: (营收 + 补贴 - 总成本) / 总成本 0.08
191     #  $\rightarrow \text{营收} + \text{补贴} \quad 1.08 \times \text{总成本}$ 
192     #  $\rightarrow \sum demand \cdot p_s \cdot 12 \quad 1.08 \times \text{total\_cost\_base} - \text{subsidy\_pool\_annual}$ 
193     max_allowed_revenue = 1.08 * total_cost_base - subsidy_pool_annual
194     prob += ann_rev_expr <= max_allowed_revenue, f"MarginCap_{st_name}"
195
196     # 非负利润 (可持续性)
197     prob += ann_rev_expr + subsidy_pool_annual >= total_cost_base, f"NonNegProfit_{st_name}"
198
199     print(f"\n[约束] {st_name}: 总成本基线={total_cost_base/10000:.1f}万, "
200           f"补贴池={subsidy_pool_annual/10000:.1f}万, "
201           f"营收上限={max_allowed_revenue/10000:.1f}万")
202
203 # ---- 3d. 目标函数: 词典序 (主:  $S3$ 最大化, 次: 营收最大化) ----
204 # 综合满意度  $S = 0.2 \cdot S1 + 0.3 \cdot S2 + 0.5 \cdot S3$ 
205 #  $S1, S2$  已由  $Q2$  固定, 故等价于最大化  $\sum demand_s \times S3_s$ 
206 # 次要目标: 在同等  $S3$  水平下, 偏好更高营收 (保证定价不低于必要水平)
207 primary_obj = pulp.lpSum([
208     total_demand_by_svc[svc] * s3[svc] * 0.5
209     for svc in services
210 ])
211 # 次要目标: 营收项 (权重极小, 仅用于打破  $S3$  平坦区域的对称性)
212 secondary_obj = 0.0001 * pulp.lpSum([
213     total_demand_by_svc[svc] * p[svc] * 12
214     for svc in services
215 ])
216 prob += primary_obj + secondary_obj
217

```

```

218 print(f"\n[MILP] 变量={len(prob.variables())}, 约束={len(prob.constraints)}")
219
220 # =====
221 # S4 求解 (含三级求解器级联备用)
222 # =====
223 # Q3 模型极轻量 (<40变量), CBC默认求解即秒级收敛
224 # 备用接口预置以应对未来扩展至区级规划 (n>100) 的大规模场景
225 try:
226     prob.solve(pulp.PULP_CBC_CMD(msg=False, timeLimit=60))
227 except Exception:
228     print("[Solver] CBC失败, 尝试Gurobi...")
229     try:
230         import gurobipy
231         prob.solve(pulp.GUROBI(msg=False, timeLimit=60))
232     except (ImportError, Exception):
233         print("[Solver] Gurobi不可用. 请检查授权或联系管理员算号.")
234         raise
235 print(f"状态: {pulp.LpStatus[prob.status]}, 目标={pulp.value(prob.objective):.4f}")
236
237 # =====
238 # S5 结果提取与导出
239 # =====
240 print("\n" + "=" * 60)
241 print("$5 Q3 最优定价方案")
242 print("=" * 60)
243
244 results = []
245 for svc in services:
246     p_opt = pulp.value(p[svc]) if svc != '紧急救助' else 0.0
247     s3_opt = pulp.value(s3[svc])
248     base = revenue_base[svc]
249     ratio = p_opt / base if base > 0 else 0.0
250
251     segment_name = ''
252     if svc == '紧急救助':
253         segment_name = '公益免费'
254     elif ratio <= 1.0:
255         segment_name = '平价'
256     elif ratio <= 1.1:
257         segment_name = '微溢价'
258     elif ratio <= 1.2:
259         segment_name = '中溢价'
260     else:
261         segment_name = '高溢价'
262
263     row = {
264         '服务项目': svc,
265         '基准价(元)': base,
266         '最优定价(元)': round(p_opt, 2),
267         '溢价率(%)': round((ratio - 1) * 100, 1) if base > 0 else 0,
268         '价格区间': segment_name,
269         'S3(价格满意度)': round(s3_opt, 3),
270         '月需求(次)': round(total_demand_by_svc[svc], 0),
271     }
272     results.append(row)
273 print(f" {svc}: 基准{base}元 → 最优{row['最优定价(元)']}元 ")

```

```

274         f"({row['价格区间']}, S3={s3_opt:.3f}), 月需求={total_demand_by_svc[svc]:.0f}次")
275
276 # ---- 站点利润核算 ----
277 print(f"\n{'='*60}")
278 print("$5.1 最优定价下的站点运营指标")
279 print(f"{'='*60}")
280
281 profit_rows = []
282 for st_name in ['F', 'H', 'J']:
283     sz = STATION_SIZES[st_name]
284     ann_rev = sum(station_demand[st_name][svc] * pulp.value(p[svc]) * 12 for svc in services
285 )
286     ann_direct = sum(station_demand[st_name][svc] * cost_per_svc[svc] * 12 for svc in
287 services)
288     daily_non_em = sum(station_demand[st_name][svc] for svc in services if svc != '紧急救助'
289 ) / MONTHLY
290     ann_sub = min(daily_non_em * 2, SUBSIDY_CAP[sz]) * ANNUAL_DAYS
291     ann_op = daily_op_arr[sz] * ANNUAL_DAYS
292     ann_dep = build_cost_arr[sz] * 10000 / 20
293     total_cost = ann_op + ann_dep + ann_direct
294     profit = ann_rev + ann_sub - total_cost
295     margin = profit / total_cost * 100
296
297 # 计算加权 S (S1/S2 来自 Q2)
298 # Q2: S1 per assignment, S2 per station
299 # 用 Q2 的值做固定输入
300 s1_map = {'A': 0.900, 'B': 0.900, 'C': 0.600, 'D': 0.900, 'E': 0.600,
301 'F': 1.000, 'G': 0.750, 'H': 1.000, 'I': 0.600, 'J': 1.000}
302 s2_map = {'F': 0.600, 'H': 0.600, 'J': 0.600}
303
304 total_weighted_s = 0
305 total_demand_count = 0
306 for comm, assigned_st in ASSIGNMENTS.items():
307     if assigned_st == st_name:
308         for svc in services:
309             d = df_demand[(df_demand["小区"] == comm) & (df_demand["服务项目"] == svc)][
310 "实际月需求(次)"].sum()
311             s_val = 0.2 * s1_map[comm] + 0.3 * s2_map[st_name] + 0.5 * pulp.value(s3[svc
312 ])
313             total_weighted_s += d * s_val
314             total_demand_count += d
315
316 avg_s = total_weighted_s / total_demand_count if total_demand_count > 0 else 0
317
318 profit_rows.append({
319     '站点': st_name,
320     '规模': size_label[sz],
321     '年营收(万元)': round(ann_rev / 10000, 2),
322     '年补贴(万元)': round(ann_sub / 10000, 2),
323     '年总成本(万元)': round(total_cost / 10000, 2),
324     '年利润(万元)': round(profit / 10000, 2),
325     '利润率(%)': round(margin, 1),
326     '加权综合满意度': round(avg_s, 3),
327 })
328
329 print(f" {st_name}({size_label[sz]}): 营收={ann_rev/10000:.1f}万, "

```

```

325         f" 补贴={ann_sub/10000:.1f}万，成本={total_cost/10000:.1f}万，"
326         f" 利润={profit/10000:.1f}万，利润率={margin:.1f}%，加权S={avg_s:.3f}")
327
328 # ---- 导出 ----
329 df_pricing = pd.DataFrame(results)
330 df_pricing.to_csv("../results/q3_optimal_pricing.csv", index=False, encoding="utf-8-sig")
331 print(f"\n[导出] results/q3_optimal_pricing.csv")
332
333 df_profit = pd.DataFrame(profit_rows)
334 df_profit.to_csv("../results/q3_station_profit.csv", index=False, encoding="utf-8-sig")
335 print(f"[导出] results/q3_station_profit.csv")
336
337 # ---- 交叉补贴分析 ----
338 print(f"\n{' '*60}")
339 print("$5.2 交叉补贴机制分析")
340 print(f"{' '*60}")
341 for st_name in ['F', 'H', 'J']:
342     emergency_demand = station_demand[st_name]['紧急救助']
343     emergency_cost_annual = emergency_demand * 8 * 12 # 紧急救助无营收、无补贴
344     print(f"    {st_name}: 紧急救助月需求={emergency_demand:.0f}次，"
345           f" 年净支出={emergency_cost_annual/10000:.2f}万（由营利性服务交叉补贴）")
346
347 # 各站点有效补贴率
348 print(f"\n    有效补贴率（受日上限约束）:")
349 for st_name in ['F', 'H', 'J']:
350     sz = STATION_SIZES[st_name]
351     daily_non_em = sum(station_demand[st_name][svc] for svc in services if svc != '紧急救助'
352                        ) / MONTHLY
353     raw_sub = daily_non_em * 2
354     actual_sub = min(raw_sub, SUBSIDY_CAP[sz])
355     effective_rate = actual_sub / daily_non_em if daily_non_em > 0 else 0
356     print(f"    {st_name}({size_label[sz]}): 理论{raw_sub:.0f}元/日 → "
357           f" 实际{actual_sub:.0f}元/日（上限{SUBSIDY_CAP[sz]}），"
358           f" 有效补贴率={effective_rate:.2f}元/次")
359 print("\nQ3 完成")

```

F Q4 灵敏度分析与鲁棒性检验代码

文件: code/solve_q4_sensitivity.py (654 行, 含三场景参数扫描 + Q1-Q2 全链路重求解 + 韧性矩阵 + 可视化导出)

Listing 7: solve_q4_sensitivity.py — 三场景灵敏度分析（完整代码）

```

1 """
2 solve_q4_sensitivity.py — Q4 灵敏度分析与鲁棒性检验（电工杯 B 题 2026）
3 =====
4 Stage 06 | 三场景参数扫描：
5     场景A：预算放松（120→130→140→150万） — 财政政策杠杆
6     场景B：成本膨胀（建设+运营成本 × 1.20） — 供给侧冲击
7     场景C：银发海啸（半失能转移 4.5%→5.5%，失能转移 10%→12%） — 需求侧冲击
8
9 输出：results/q4_sensitivity_summary.csv + 可视化对比
10 """

```



```

11
12 import numpy as np
13 import pandas as pd
14 import pulp
15 import matplotlib
16 matplotlib.use('Agg')
17 import matplotlib.pyplot as plt
18 import matplotlib.font_manager as fm
19 import seaborn as sns
20 import os, warnings, copy, glob
21 warnings.filterwarnings('ignore')
22
23 # =====
24 # 字体注册 (必须在任何绘图前)
25 # =====
26 def register_chinese_font():
27     cache_dir = matplotlib.get_cachedir()
28     for f in glob.glob(os.path.join(cache_dir, '*font*')):
29         try: os.remove(f)
30         except: pass
31     fm.fontManager.addfont("C:/Windows/Fonts/simhei.ttf")
32     fm.fontManager.addfont("C:/Windows/Fonts/simkai.ttf")
33     fm.fontManager.addfont("C:/Windows/Fonts/simsun.ttc")
34     fm._load_fontmanager(try_read_cache=False)
35     registered = [f.name for f in fm.fontManager.ttflist]
36     return 'SimHei' if 'SimHei' in registered else registered[0]
37
38 FONT_NAME = register_chinese_font()
39
40 sns.set_style("whitegrid")
41 plt.rcParams.update({
42     "font.family": "sans-serif",
43     "font.sans-serif": [FONT_NAME, "Microsoft YaHei", "SimHei", "DejaVu Sans"],
44     "axes.unicode_minus": False,
45     "font.size": 16,
46     "axes.titlesize": 18,
47     "axes.labelsize": 15,
48     "legend.fontsize": 14,
49     "xtick.labelsize": 13,
50     "ytick.labelsize": 13,
51     "figure.dpi": 300,
52     "savefig.dpi": 300,
53     "savefig.bbox": "tight",
54     "axes.spines.top": False,
55     "axes.spines.right": False,
56 })
57 print(f"[Font] Registered: {FONT_NAME}")
58
59 BASE = os.path.join(os.path.dirname(__file__), "..", "data")
60 os.makedirs("../results", exist_ok=True)
61 os.makedirs("../figures", exist_ok=True)
62
63 # =====
64 # S1 数据加载
65 # =====
66 df_pop_orig = pd.read_csv("../results/q1_final_population.csv")

```

```

67 communities = df_pop_orig["小区"].tolist()
68 n_comm = len(communities)
69
70 df_demand_orig = pd.read_csv("../results/q1_service_demand.csv")
71
72 df_dist = pd.read_excel(os.path.join(BASE, "附件4: 小区间距离矩阵.xlsx"),
73                         sheet_name="小区间距离矩阵", skiprows=1)
74 df_dist.columns = ["小区"] + communities
75 df_dist = df_dist.dropna(subset=["小区"]).set_index("小区")
76 dist_matrix = df_dist.values.astype(float)
77 reachable = (dist_matrix <= 1000)
78
79 # =====
80 # S2 基线参数
81 # =====
82 BASELINE = {
83     'budget': 120.0,
84     'build_cost': np.array([18, 32, 45], dtype=float),
85     'daily_op': np.array([2000, 3200, 4400], dtype=float),
86     'daily_cap': np.array([1000, 2000, 3000], dtype=float),
87     'alpha_sm': 0.045,    # 自理→半失能
88     'alpha_md': 0.10,    # 半失能→失能
89     'mu': 0.05,          # 死亡率
90     'beta': 0.07,        # 新增率
91     'consumption_caps': [0.20, 0.25, 0.30],
92     'radius': 1000,
93     'monthly': 30,
94 }
95
96 # =====
97 # S3 Q1 人口预测函数 (可参数化)
98 # =====
99 def build_transition_matrix(mu=0.05, alpha_sm=0.045, alpha_md=0.10, beta=0.07):
100     A = np.array([
101         [(1 - mu) * (1 - alpha_sm) + beta,    beta,                    beta],
102         [(1 - mu) * alpha_sm,                  (1 - mu) * (1 - alpha_md), 0.0],
103         [0.0,                                  (1 - mu) * alpha_md,        (1 - mu)],
104     ])
105     return A
106
107 def predict_population_from_original(A, T=5):
108     """从原始附件1读取初始人口, 用自定义A预测5年."""
109     df_orig = pd.read_excel(
110         os.path.join(BASE, "附件1: 小区基础数据.xlsx"),
111         sheet_name="人口与老人结构", skiprows=1
112     )
113     df_orig.columns = ["小区", "总人口", "60plus", "自理", "半失能", "失能", "人均月收入"]
114     S0 = df_orig[["自理", "半失能", "失能"]].values.astype(np.float64)
115     income = df_orig["人均月收入"].values.astype(np.float64)
116     communities_list = df_orig["小区"].tolist()
117
118     S_hist = np.zeros((T+1, len(communities_list), 3))
119     S_hist[0] = S0.copy()
120     for t in range(T):
121         for i in range(len(communities_list)):
122             S_hist[t+1, i] = np.round(A @ S_hist[t, i])

```

```

123     return S_hist[-1], income, communities_list
124
125 def compute_demand_from_final(S_final, income, consumption_caps, communities_list):
126     """计算 t=5 实际服务需求."""
127     # 读取每位老人月均需求
128     df_req = pd.read_excel(
129         os.path.join(BASE, "附件2: 服务需求数据.xlsx"),
130         sheet_name="每位老人月均服务需求次数", skiprows=1
131     )
132     df_req.columns = ["服务项目", "自理", "半自理", "失能"]
133     demand_per_capita = df_req[["自理", "半自理", "失能"]].values.astype(np.float64)
134     services = df_req["服务项目"].tolist()
135     revenue = np.array([10, 20, 30, 28, 25, 0], dtype=np.float64)
136     cost = np.array([8, 16, 24, 23, 20, 8], dtype=np.float64)
137
138     n_comm, n_type = S_final.shape
139     n_svc = demand_per_capita.shape[0]
140
141     theoretical = np.zeros((n_comm, n_type, n_svc))
142     for i in range(n_comm):
143         for e in range(n_type):
144             theoretical[i, e, :] = S_final[i, e] * demand_per_capita[:, e]
145
146     caps = np.array(consumption_caps)
147     actual = np.zeros_like(theoretical)
148     for i in range(n_comm):
149         for e in range(n_type):
150             full_cost = np.sum(demand_per_capita[:, e] * revenue)
151             budget = income[i] * caps[e]
152             ratio = min(1.0, budget / full_cost) if full_cost > 0 else 1.0
153             actual[i, e, :] = theoretical[i, e, :] * ratio
154
155     # 构建 DataFrame 格式需求数据
156     rows = []
157     for i, comm in enumerate(communities_list):
158         for e, etype in enumerate(["自理", "半失能", "失能"]):
159             for s, svc in enumerate(services):
160                 rows.append({
161                     "小区": comm,
162                     "老人类型": etype,
163                     "服务项目": svc,
164                     "理论月需求(次)": round(theoretical[i, e, s]),
165                     "实际月需求(次)": round(actual[i, e, s]),
166                     "单次营收(元)": revenue[s],
167                     "单次成本(元)": cost[s],
168                 })
169     return pd.DataFrame(rows), services, revenue, cost
170
171 # =====
172 # S4 Q2 MILP 求解函数 (可参数化)
173 # =====
174 def solve_q2_milp(df_demand, budget, build_cost, daily_op, daily_cap,
175                  time_limit=180, verbose=False):
176     """返回: {stations, assignments, coverage, objective, profit_margins, key_metrics}"""
177     communities = sorted(df_demand["小区"].unique())
178     n_comm = len(communities)

```

```

179
180 actual_demand = np.array([df_demand[df_demand["小区"]==c]["实际月需求(次)"].sum() for c
    in communities])
181
182 n_sizes = 3
183 MONTHLY = 30
184
185 prob = pulp.LpProblem("Q4_MILP", pulp.LpMaximize)
186
187 y = pulp.LpVariable.dicts("y", [(i,k) for i in range(n_comm) for k in range(n_sizes)],
    cat=pulp.LpBinary)
188 x = pulp.LpVariable.dicts("x", [(i,j) for i in range(n_comm) for j in range(n_comm)
    if i==j or reachable[i,j]], cat=pulp.LpBinary)
189
190 s1 = pulp.LpVariable.dicts("s1", [(i,j) for i in range(n_comm) for j in range(n_comm)
    if i==j or reachable[i,j]], lowBound=0, upBound=1)
191
192 s2 = pulp.LpVariable.dicts("s2", range(n_comm), lowBound=0, upBound=1)
193 s = pulp.LpVariable.dicts("s", [(i,j) for i in range(n_comm) for j in range(n_comm)
    if i==j or reachable[i,j]], lowBound=0, upBound=1)
194
195
196 delta_s1 = {}
197 for i in range(n_comm):
198     for j in range(n_comm):
199         if i==j or reachable[i,j]:
200             for ell in range(4):
201                 delta_s1[(i,j,ell)] = pulp.LpVariable(f"d1_{i}_{j}_{ell}", cat=pulp.
                    LpBinary)
202
203 delta_s2 = {}
204 for i in range(n_comm):
205     for ell in range(5):
206         delta_s2[(i,ell)] = pulp.LpVariable(f"d2_{i}_{ell}", cat=pulp.LpBinary)
207
208 w = pulp.LpVariable.dicts("w", [(i,j) for i in range(n_comm) for j in range(n_comm)
    if i==j or reachable[i,j]], lowBound=0, upBound=1)
209
210 station_exists = pulp.LpVariable.dicts("z", range(n_comm), cat=pulp.LpBinary)
211
212 # Budget
213 prob += pulp.lpSum([y[(i,k)] * build_cost[k] for i in range(n_comm) for k in range(
    n_sizes)]) <= budget
214
215 for i in range(n_comm):
216     prob += pulp.lpSum([y[(i,k)] for k in range(n_sizes)]) == station_exists[i]
217
218 for i in range(n_comm):
219     for j in range(n_comm):
220         if i==j or reachable[i,j]:
221             prob += x[(i,j)] <= station_exists[i]
222
223 for j in range(n_comm):
224     can_serve = [i for i in range(n_comm) if i==j or reachable[i,j]]
225     prob += pulp.lpSum([x[(i,j)] for i in can_serve]) <= 1
226
227 for i in range(n_comm):
228     prob += x[(i,i)] >= station_exists[i]
229
230 for i in range(n_comm):

```

```

231     for j in range(n_comm):
232         if i==j or reachable[i,j]:
233             prob += w[(i,j)] <= x[(i,j)]
234             prob += w[(i,j)] <= s2[i]
235             prob += w[(i,j)] >= s2[i] - (1 - x[(i,j)])
236
237     for i in range(n_comm):
238         monthly_cap = pulp.lpSum([y[(i,k)] * daily_cap[k] * MONTHLY for k in range(n_sizes)
239                                     ])
240         effective_load = pulp.lpSum([
241             actual_demand[j] * s[(i,j)]
242             for j in range(n_comm) if i==j or reachable[i,j]
243         ])
244         prob += effective_load <= monthly_cap + 1e6 * (1 - station_exists[i])
245
246     S1_lo = [0, 300, 500, 650]
247     S1_hi = [300, 500, 650, 1000]
248     S1_val = [1.00, 0.90, 0.75, 0.60]
249
250     for i in range(n_comm):
251         for j in range(n_comm):
252             if i==j:
253                 prob += s1[(i,j)] == station_exists[i]
254             elif reachable[i,j]:
255                 d = dist_matrix[i,j]
256                 prob += pulp.lpSum([delta_s1[(i,j,ell)] for ell in range(4)]) == x[(i,j)]
257                 for ell in range(4):
258                     prob += d >= S1_lo[ell] * delta_s1[(i,j,ell)]
259                     prob += d <= S1_hi[ell] + 1800 * (1 - delta_s1[(i,j,ell)])
260                     prob += s1[(i,j)] == pulp.lpSum([S1_val[ell] * delta_s1[(i,j,ell)] for ell
261                                                         in range(4)])
262
263     S2_lo = [0.0, 0.60, 0.75, 0.85, 0.95]
264     S2_hi = [0.60, 0.75, 0.85, 0.95, 1.00]
265     S2_val = [1.00, 0.93, 0.85, 0.72, 0.60]
266
267     for i in range(n_comm):
268         prob += pulp.lpSum([delta_s2[(i,ell)] for ell in range(5)]) == station_exists[i]
269         effective_load_i = pulp.lpSum([
270             actual_demand[j] * s[(i,j)]
271             for j in range(n_comm) if i==j or reachable[i,j]
272         ])
273         capacity_i = pulp.lpSum([y[(i,k)] * daily_cap[k] * MONTHLY for k in range(n_sizes)])
274         for ell in range(5):
275             prob += effective_load_i >= S2_lo[ell] * capacity_i - 1e6 * (1 - delta_s2[(i,ell)
276                                                         ])
277             prob += effective_load_i <= S2_hi[ell] * capacity_i + 1e6 * (1 - delta_s2[(i,ell)
278                                                         ])
279         prob += s2[i] == pulp.lpSum([S2_val[ell] * delta_s2[(i,ell)] for ell in range(5)])
280
281     for i in range(n_comm):
282         for j in range(n_comm):
283             if i==j or reachable[i,j]:
284                 prob += s[(i,j)] == 0.2 * s1[(i,j)] + 0.3 * w[(i,j)] + 0.5 * x[(i,j)]
285                 prob += s[(i,j)] >= 0.6 * x[(i,j)]
286                 prob += s[(i,j)] <= 1.0 * x[(i,j)]

```

```

283
284 coverage_count = pulp.lpSum([x[(i,j)] for i in range(n_comm) for j in range(n_comm)
285                               if i==j or reachable[i,j]])
286 total_sat = pulp.lpSum([s[(i,j)] for i in range(n_comm) for j in range(n_comm)
287                          if i==j or reachable[i,j]])
288 prob += 5.0 * coverage_count + 0.5 * total_sat
289
290 if verbose:
291     print(f" 变量={len(prob.variables())}, 约束={len(prob.constraints)}, 求解中...")
292 prob.solve(pulp.PULP_CBC_CMD(msg=False, timeLimit=time_limit, gapRel=0.01,
293                               options=['randomSeed=42']))
294
295 # 提取结果
296 built = []
297 for i in range(n_comm):
298     for k in range(n_sizes):
299         if pulp.value(y[(i,k)]) > 0.5:
300             built.append({'loc': communities[i], 'size': k})
301
302 assignments = {}
303 for j in range(n_comm):
304     for i in range(n_comm):
305         if (i==j or reachable[i,j]) and pulp.value(x.get((i,j), 0)) > 0.5:
306             assignments[communities[j]] = communities[i]
307             break
308
309 coverage = len(assignments)
310 obj = pulp.value(prob.objective)
311 status = pulp.LpStatus[prob.status]
312
313 # 利润计算
314 revenue_map = {'助餐':10,'日间照料':20,'上门护理':30,'康复理疗':28,'助浴':25,'紧急救助':0}
315 cost_map = {'助餐':8,'日间照料':16,'上门护理':24,'康复理疗':23,'助浴':20,'紧急救助':8}
316
317 profit_margins = {}
318 total_profit = 0
319 for st in built:
320     i = communities.index(st['loc'])
321     ann_rev = ann_cost_val = ann_sub = 0
322     for j_comm, assigned_st in assignments.items():
323         if assigned_st == st['loc']:
324             j = communities.index(j_comm)
325             mask = df_demand["小区"] == j_comm
326             for _, row in df_demand[mask].iterrows():
327                 svc = row["服务项目"]
328                 d = row["实际月需求(次)"]
329                 ann_rev += d * revenue_map.get(svc,0) * 12
330                 ann_cost_val += d * cost_map.get(svc,0) * 12
331                 if svc != "紧急救助":
332                     ann_sub += d * 2 * 12
333
334     k = st['size']
335     ann_op = daily_op[k] * 360
336     ann_dep = build_cost[k] * 10000 / 20
337     total_cost = ann_op + ann_dep + ann_cost_val
338     profit = ann_rev + ann_sub - total_cost

```

```

338         rate = profit / total_cost * 100 if total_cost > 0 else 0
339         profit_margins[st['loc']] = {'profit': profit/10000, 'rate': rate}
340         total_profit += profit
341
342     # 计算加权满意度
343     s1_map_q2 = {}
344     s2_map_q2 = {}
345     s_map = {}
346     for j_comm, i_comm in assignments.items():
347         j = communities.index(j_comm)
348         i = communities.index(i_comm)
349         if (i==j or reachable[i,j]):
350             sv = pulp.value(s.get((i,j), 0))
351             s1v = pulp.value(s1.get((i,j), 0))
352             s2v = pulp.value(s2[i])
353             s_map[j_comm] = sv
354             s1_map_q2[j_comm] = s1v
355             s2_map_q2[i_comm] = s2v
356
357     avg_satisfaction = np.mean(list(s_map.values())) if s_map else 0
358
359     return {
360         'status': status,
361         'objective': obj,
362         'coverage': coverage,
363         'n_communities': n_comm,
364         'built_stations': built,
365         'assignments': assignments,
366         'profit_margins': profit_margins,
367         'total_profit': total_profit / 10000,
368         'avg_satisfaction': avg_satisfaction,
369         'uncovered': [c for c in communities if c not in assignments],
370         'budget_used': sum(build_cost[s['size']] for s in built),
371         'satisfaction_map': s_map,
372         's2_map': s2_map_q2,
373     }
374
375 # =====
376 # S5 场景定义与批量求解
377 # =====
378 print("=" * 70)
379 print("Q4 灵敏度分析 — 三场景参数扫描")
380 print("=" * 70)
381
382 # ---- 基线：直接从 Q2 JSON 加载，保证与论文正文 54.225 完全一致 ----
383 import json as _json
384 print("\n[基线] 加载 Q2 确定性最优解 (F大型+H中型+J中型, Obj=54.225)...")
385 with open("../results/q2_baseline.json", "r", encoding="utf-8") as _f:
386     _q2 = _json.load(_f)
387 baseline = {
388     'status': 'Optimal (from Q2 JSON)',
389     'objective': _q2['objective'],
390     'coverage': _q2['coverage'],
391     'n_communities': _q2['n_communities'],
392     'built_stations': [{'loc': s['loc'], 'size': s['size']} for s in _q2['stations']],
393     'assignments': {k: v['station'] for k, v in _q2['assignments'].items()},

```

```

394     'profit_margins': {s['loc']: {'profit': s['profit'], 'rate': s['rate']}}
395         for s in _q2['stations']],
396     'total_profit': sum(s['profit'] for s in _q2['stations']),
397     'avg_satisfaction': _q2['avg_satisfaction'],
398     'uncovered': [],
399     'budget_used': _q2['budget_used'],
400     'satisfaction_map': {k: v['S'] for k, v in _q2['assignments'].items()},
401     's2_map': {s['loc']: _q2['s2_by_station'].get(s['loc'], 0.600) for s in _q2['stations']
402     ]},
403 }
404 print(f" 目标={baseline['objective']:.4f}, 覆盖={baseline['coverage']}/{baseline['n_communities']}, "
405       f"利润={baseline['total_profit']:.1f}万/年, 平均S={baseline['avg_satisfaction']:.4f}")
406 print(f" 站点: {[s['loc'], ['小', '中', '大'][s['size']]] for s in baseline['built_stations']}]")
407 print(f" 确认] 与 Q2 正文数据完全一致 (F大型+H中型+J中型, 109万元, 10/10覆盖)")
408 # ---- 场景A: 人口结构冲击 (题目4.1第一条: 增长率/转移概率同时变化) ----
409 print("\n" + "-"*50)
410 print(f"[场景A] 人口结构冲击: 增长率7%→8%, _sm 4.5%→5.5%, _md 10%→9.5%")
411 print("-"*50)
412 A_pop = build_transition_matrix(mu=BASELINE['mu'], alpha_sm=0.055,
413                                alpha_md=0.095, beta=0.08)
414 S_final_pop, income_pop, comm_list = predict_population_from_original(A_pop, T=5)
415 total_pop = S_final_pop.sum()
416 disabled_pop = S_final_pop[:, 2].sum()
417 disabled_pct_pop = disabled_pop / total_pop * 100
418 print(f" 人口预测: 60+总人口={total_pop:.0f} (基线7577), "
419       f"失能={disabled_pop:.0f} (基线1125), 失能率={disabled_pct_pop:.1f}% (基线14.8%)")
420 df_demand_pop, _, _, _ = compute_demand_from_final(
421     S_final_pop, income_pop, BASELINE['consumption_caps'], comm_list)
422 scenario_a = solve_q2_milp(df_demand_pop, BASELINE['budget'],
423                            BASELINE['build_cost'], BASELINE['daily_op'],
424                            BASELINE['daily_cap'], time_limit=90)
425 print(f" 覆盖={scenario_a['coverage']}/{scenario_a['n_communities']}, "
426       f"目标={scenario_a['objective']:.2f}, 利润={scenario_a['total_profit']:.1f}万/年, "
427       f"平均S={scenario_a['avg_satisfaction']:.3f}")
428 print(f" 站点: {[s['loc'], ['小', '中', '大'][s['size']]] for s in scenario_a['built_stations']}]")
429 print(f" 未覆盖: {scenario_a['uncovered']}, 已用预算={scenario_a['budget_used']}万")
430
431 # ---- 场景B: 日固定管理成本+20% (题目4.1第二条: 仅运营成本上浮) ----
432 print("\n" + "-"*50)
433 print(f"[场景B] 成本冲击: 日固定管理成本+20% (建设成本不变)")
434 print("-"*50)
435 inflated_daily_op_only = BASELINE['daily_op'] * 1.20
436 scenario_b = solve_q2_milp(df_demand_orig, BASELINE['budget'],
437                            BASELINE['build_cost'], inflated_daily_op_only,
438                            BASELINE['daily_cap'], time_limit=90)
439 print(f" 覆盖={scenario_b['coverage']}/{scenario_b['n_communities']}, "
440       f"目标={scenario_b['objective']:.2f}, 利润={scenario_b['total_profit']:.1f}万/年, "
441       f"平均S={scenario_b['avg_satisfaction']:.3f}")
442 print(f" 站点: {[s['loc'], ['小', '中', '大'][s['size']]] for s in scenario_b['built_stations']}]")
443 print(f" 未覆盖: {scenario_b['uncovered']}, 已用预算={scenario_b['budget_used']}万")
444

```



```

445 # ---- 场景C: 预算调整至140万 (题目4.1第三条) ----
446 print("\n" + "-"*50)
447 print("[场景C] 预算调整: 总建设预算120-140万")
448 print("-"*50)
449 scenario_c = solve_q2_milp(df_demand_orig, 140.0,
450                             BASELINE['build_cost'], BASELINE['daily_op'],
451                             BASELINE['daily_cap'], time_limit=90)
452 print(f" 覆盖={scenario_c['coverage']}/{scenario_c['n_communities']}, "
453       f"目标={scenario_c['objective']:.2f}, 利润={scenario_c['total_profit']:.1f}万/年, "
454       f"平均S={scenario_c['avg_satisfaction']:.3f}")
455 print(f" 站点: {[s['loc'], ['小','中','大'][s['size']]] for s in scenario_c['
    built_stations']]}")
456 print(f" 未覆盖: {scenario_c['uncovered']}, 已用预算={scenario_c['budget_used']}万")
457
458 # =====
459 # $6 汇总与导出
460 # =====
461 print("\n" + "=" * 70)
462 print("$6 灵敏度汇总表")
463 print("=" * 70)
464
465 summary_rows = []
466
467 # 基线行
468 summary_rows.append({
469     '场景': '基线 (Baseline)',
470     '预算(万元)': 120,
471     '成本系数': '1.00',
472     '60+人口': 7577,
473     '失能人口': 1125,
474     '失能占比(%)': 14.8,
475     '覆盖数': baseline['coverage'],
476     '覆盖率(%)': round(baseline['coverage']/baseline['n_communities']*100, 1),
477     '站点配置': '+'.join([f"{s['loc']}({'小','中','大')[s['size']]}" for s in baseline['
        built_stations']]),
478     '已用预算(万元)': baseline['budget_used'],
479     '目标值': round(baseline['objective'], 2),
480     '平均S': round(baseline['avg_satisfaction'], 3),
481     '年总利润(万元)': round(baseline['total_profit'], 1),
482     '未覆盖小区': ','.join(baseline['uncovered']) if baseline['uncovered'] else '无',
483 })
484
485 # 场景A: 人口结构冲击
486 summary_rows.append({
487     '场景': 'A: 人口结构冲击 (r=8%, _sm=5.5%, _md=9.5%)',
488     '预算(万元)': 120,
489     '成本系数': '1.00',
490     '60+人口': int(total_pop),
491     '失能人口': int(disabled_pop),
492     '失能占比(%)': round(disabled_pct_pop, 1),
493     '覆盖数': scenario_a['coverage'],
494     '覆盖率(%)': round(scenario_a['coverage']/scenario_a['n_communities']*100, 1),
495     '站点配置': '+'.join([f"{s['loc']}({'小','中','大')[s['size']]}" for s in scenario_a['
        built_stations']]),
496     '已用预算(万元)': scenario_a['budget_used'],
497     '目标值': round(scenario_a['objective'], 2),

```

```

498     '平均S': round(scenario_a['avg_satisfaction'], 3),
499     '年总利润(万元)': round(scenario_a['total_profit'], 1),
500     '未覆盖小区': ', '.join(scenario_a['uncovered']) if scenario_a['uncovered'] else '无',
501 })
502
503 # 场景B: 运营成本+20%
504 summary_rows.append({
505     '场景': 'B: 日固定管理成本+20%',
506     '预算(万元)': 120,
507     '成本系数': '1.20(op only)',
508     '60+人口': 7577,
509     '失能人口': 1125,
510     '失能占比(%)': 14.8,
511     '覆盖数': scenario_b['coverage'],
512     '覆盖率(%)': round(scenario_b['coverage']/scenario_b['n_communities']*100, 1),
513     '站点配置': '+'.join([f"{s['loc']}({['小','中','大'][s['size']])}" for s in scenario_b['
        built_stations']]),
514     '已用预算(万元)': scenario_b['budget_used'],
515     '目标值': round(scenario_b['objective'], 2),
516     '平均S': round(scenario_b['avg_satisfaction'], 3),
517     '年总利润(万元)': round(scenario_b['total_profit'], 1),
518     '未覆盖小区': ', '.join(scenario_b['uncovered']) if scenario_b['uncovered'] else '无',
519 })
520
521 # 场景C: 预算140万
522 summary_rows.append({
523     '场景': 'C: 预算调整至140万',
524     '预算(万元)': 140,
525     '成本系数': '1.00',
526     '60+人口': 7577,
527     '失能人口': 1125,
528     '失能占比(%)': 14.8,
529     '覆盖数': scenario_c['coverage'],
530     '覆盖率(%)': round(scenario_c['coverage']/scenario_c['n_communities']*100, 1),
531     '站点配置': '+'.join([f"{s['loc']}({['小','中','大'][s['size']])}" for s in scenario_c['
        built_stations']]),
532     '已用预算(万元)': scenario_c['budget_used'],
533     '目标值': round(scenario_c['objective'], 2),
534     '平均S': round(scenario_c['avg_satisfaction'], 3),
535     '年总利润(万元)': round(scenario_c['total_profit'], 1),
536     '未覆盖小区': ', '.join(scenario_c['uncovered']) if scenario_c['uncovered'] else '无',
537 })
538
539 df_summary = pd.DataFrame(summary_rows)
540 df_summary.to_csv("../results/q4_sensitivity_summary.csv", index=False, encoding="utf-8-sig"
541 )
542
543 # LaTeX 表格
544 print("\n" + "=" * 70)
545 print("LaTeX 灵敏度汇总表 (可直接粘贴到 paper_workspace)")
546 print("=" * 70)
547 print(r"""
548 \begin{table}[htbp]
549 \centering
550 \caption{三场景参数扫描灵敏度汇总}

```

```

551 \label{tab:q4_sensitivity}
552 \small
553 \begin{tabular}{lcccccc}
554 \hline
555 \textbf{场景} & \textbf{预算(万)} & \textbf{覆盖率} & \textbf{站点配置} & \textbf{目标值} & \\
556 & \textbf{平均S} & \textbf{年利润(万)} & \\\
557 \hline""")
558 for _, row in df_summary.iterrows():
559     print(f"{row['场景']} & {row['预算(万元)']} & {row['覆盖率(%)']}\% & "
560           f"{row['站点配置']} & {row['目标值']:.2f} & {row['平均S']:.3f} & {row['年总利润(万'
561           元')]:.1f} \\\")
562 print(r""\hline
563 \end{tabular}
564 \end{table}
565 """)
566 # =====
567 # S7 可视化：灵敏度对比图 (v4: seaborn muted 学术风格)
568 # =====
569 sns.set_style("white")
570 # 确保字体在 seaborn 样式重置后仍然生效
571 plt.rcParams.update({
572     "font.family": "sans-serif",
573     "font.sans-serif": [FONT_NAME, "Microsoft YaHei", "SimHei", "DejaVu Sans"],
574     "axes.unicode_minus": False,
575 })
576 NAVY = '#1B2A4A'
577 CYAN = '#2E86AB'
578 CORAL = '#E85D75'
579 TEAL = '#0D7377'
580 scenarios_labels = ['基线\n(原方案)', 'A-人口\n结构冲击', 'B-成本\n管理+20%', 'C-预算\n调整'
581                    '140万']
582 bar_colors = [NAVY, CYAN, CORAL, TEAL]
583 coverage_vals = [
584     baseline['coverage']/baseline['n_communities']*100,
585     scenario_a['coverage']/10*100,
586     scenario_b['coverage']/10*100,
587     scenario_c['coverage']/10*100,
588 ]
589 sat_vals = [
590     baseline['avg_satisfaction'],
591     scenario_a['avg_satisfaction'],
592     scenario_b['avg_satisfaction'],
593     scenario_c['avg_satisfaction'],
594 ]
595 profit_vals = [
596     baseline['total_profit'],
597     scenario_a['total_profit'],
598     scenario_b['total_profit'],
599     scenario_c['total_profit'],
600 ]
601 ]
602 ]
603 ]

```

```

604 # ---- 三张独立图，分别输出 ----
605
606 def make_single_fig(ax, scenario_labels, values, bar_colors, fmt, ylabel, title, ylim_bottom
    , ylim_top, hline_y=None):
607     bars = ax.bar(scenario_labels, values, color=bar_colors, edgecolor='white', lw=1.5,
        width=0.55)
608     for bar, val in zip(bars, values):
609         ax.text(bar.get_x() + bar.get_width()/2, bar.get_height() + (ylim_top-ylim_bottom)
            *0.015,
610             fmt.format(val), ha='center', fontsize=13, fontweight='bold', color='#2C3E50
                ')
611     if hline_y is not None:
612         ax.axhline(y=hline_y, color='#BDC3C7', linestyle='--', lw=1.0, alpha=0.6)
613     ax.set_title(title, fontsize=16, fontweight='bold', pad=16, color='#2C3E50')
614     ax.set_ylabel(ylabel, fontsize=13, color='#34495E')
615     ax.set_ylim(ylim_bottom, ylim_top)
616     ax.tick_params(axis='x', labelsize=11, colors='#2C3E50')
617     ax.tick_params(axis='y', labelsize=10, colors='#7F8C8D')
618     for spine in ['top', 'right', 'left', 'bottom']:
619         ax.spines[spine].set_visible(False)
620     ax.tick_params(left=False, bottom=False)
621
622 # Panel A: 覆盖率
623 fig_a, ax_a = plt.subplots(figsize=(10, 6))
624 fig_a.patch.set_facecolor('white')
625 make_single_fig(ax_a, scenarios_labels, coverage_vals, bar_colors,
626     '{:.0f}%', '覆盖率 (%)', '(a) 覆盖率', 55, 118, hline_y=100)
627 fig_a.tight_layout(pad=2.0)
628 fig_a.savefig("../figures/figure_q4_coverage.png", dpi=300, facecolor='white', bbox_inches='
    tight')
629 plt.close(fig_a)
630
631 # Panel B: 满意度
632 fig_b, ax_b = plt.subplots(figsize=(10, 6))
633 fig_b.patch.set_facecolor('white')
634 make_single_fig(ax_b, scenarios_labels, sat_vals, bar_colors,
635     '{:.3f}', '平均综合满意度 $$$', '(b) 加权满意度', 0.76, 1.03)
636 fig_b.tight_layout(pad=2.0)
637 fig_b.savefig("../figures/figure_q4_satisfaction.png", dpi=300, facecolor='white',
    bbox_inches='tight')
638 plt.close(fig_b)
639
640 # Panel C: 利润
641 fig_c, ax_c = plt.subplots(figsize=(10, 6))
642 fig_c.patch.set_facecolor('white')
643 make_single_fig(ax_c, scenarios_labels, profit_vals, bar_colors,
644     '{:.0f}', '年总利润 (万元)', '(c) 年利润', 500, max(profit_vals)*1.22)
645 fig_c.tight_layout(pad=2.0)
646 fig_c.savefig("../figures/figure_q4_profit.png", dpi=300, facecolor='white', bbox_inches='
    tight')
647 plt.close(fig_c)
648
649 print("\n[图表] figures/figure_q4_coverage.png, figure_q4_satisfaction.png, figure_q4_profit
    .png 已保存 (3张独立图)")
650
651 print("\n" + "=" * 70)

```

```

652 print("Q4 灵敏度分析完成")
653 print("=" * 70)

```

G 可视化图表生成代码

总控脚本：regenerate_all_figures.py

文件: code/regenerate_all_figures.py (统一生成论文全部 8 张图表, 含字体注册 + 配色方案 + 尺寸适配)

Listing 8: regenerate_all_figures.py 一论文全部图表统一生成脚本

```

1 """
2 regenerate_all_figures.py — Academic Noir 统一图表再生 (电工杯 B题 2026)
3 =====
4 Visio-style学术级配色: NAVY/CYAN/ROSE/AMBER/TEAL
5 生成全部论文插图 + 技术路线流程图, 风格统一, 适合学术印刷.
6
7 Usage: python regenerate_all_figures.py
8 Output: figures/ 目录下 9 张高质量 PNG (300dpi)
9 """
10
11 import numpy as np
12 import pandas as pd
13 import matplotlib
14 matplotlib.use('Agg')
15 import matplotlib.pyplot as plt
16 import matplotlib.font_manager as fm
17 import matplotlib.patches as mpatches
18 import matplotlib.lines as mlines
19 from matplotlib.patches import FancyBboxPatch, FancyArrowPatch, Arc
20 import matplotlib.ticker as ticker
21 import seaborn as sns
22 import networkx as nx
23 from pathlib import Path
24 import os, sys, json, warnings
25 warnings.filterwarnings('ignore')
26
27 # =====
28 # $O 全局样式 — Academic Noir Palette
29 # =====
30 NAVY    = '#1B2A4A'    # 主文字/深色元素
31 CYAN    = '#2E86AB'    # 自理老人 / 主强调色
32 ROSE    = '#A23B72'    # 失能老人 / 警告色
33 AMBER   = '#F18F01'    # 半失能老人 / 过渡色
34 TEAL    = '#0D7377'    # 站点标识
35 SLATE   = '#5D6D7E'    # 中性灰蓝
36 CREAM   = '#F5F0EB'    # 暖色背景
37 WHITE   = '#FFFFFF'
38 GOLD    = '#D4A843'    # 补贴/利润强调
39 CORAL   = '#E85D75'    # 预算线/阈值
40
41 PALETTE_3 = [CYAN, AMBER, ROSE]          # 自理/半失能/失能

```

```

42 PALETTE_6 = [NAVY, CYAN, TEAL, ROSE, AMBER, GOLD]
43
44 LABELS_3 = ['自理 (Self-care)', '半失能 (Semi-disabled)', '失能 (Disabled)']
45
46 # ---- 字体注册 ----
47 def register_fonts():
48     for fp in ['C:/Windows/Fonts/simhei.ttf', 'C:/Windows/Fonts/simkai.ttf',
49               'C:/Windows/Fonts/simsun.ttc', 'C:/Windows/Fonts/msyh.ttc']:
50         try: fm.fontManager.addfont(fp)
51         except: pass
52     fm._load_fontmanager(try_read_cache=False)
53     names = [f.name for f in fm.fontManager.ttflist]
54     return 'SimHei' if 'SimHei' in names else ('Microsoft YaHei' if 'Microsoft YaHei' in
55         names else names[0])
56
57 FONT = register_fonts()
58
59 def set_style():
60     sns.set_style("white")
61     plt.rcParams.update({
62         "font.family": "sans-serif",
63         "font.sans-serif": [FONT, "Microsoft YaHei", "DejaVu Sans"],
64         "axes.unicode_minus": False,
65         "font.size": 13,
66         "axes.titlesize": 15,
67         "axes.labelsize": 13,
68         "legend.fontsize": 12,
69         "xtick.labelsize": 11,
70         "ytick.labelsize": 11,
71         "figure.dpi": 300, "savefig.dpi": 300,
72         "savefig.bbox": "tight",
73         "axes.spines.top": False,
74         "axes.spines.right": False,
75         "axes.linewidth": 0.8,
76         "axes.edgecolor": SLATE,
77         "xtick.color": SLATE, "ytick.color": SLATE,
78         "grid.alpha": 0.15, "grid.color": SLATE,
79     })
80 set_style()

```

韧性热力图与雷达图：generate_extra_figures.py (核心片段)

Listing 9: generate_extra_figures.py —韧性热力图与雷达图生成

```

1 # 热力图：红(弱韧性 0.82) → 黄(中等 0.92) → 绿(强韧性 1.02)
2 cmap = sns.diverging_palette(10, 130, s=85, l=50, center='light', as_cmap=True)
3 im = ax.imshow(data, cmap=cmap, vmin=0.82, vmax=1.02, aspect='auto')
4 for i in range(len(scenarios)):
5     for j in range(len(dims)):
6         color = 'white' if data[i,j] < 0.88 else NAVY
7         ax.text(j, i, f'{data[i,j]:.3f}', ha='center', va='center',
8               fontsize=12, fontweight='bold', color=color)
9

```

```

10 # 雷达图：三场景韧性系数多边形
11 N = len(dims)
12 angles = np.linspace(0, 2*np.pi, N, endpoint=False).tolist() + [0]
13 for idx, (label, values) in enumerate(scenarios.items()):
14     vals = values + values[:1]
15     ax.fill(angles, vals, alpha=0.08, color=colors[idx])
16     ax.plot(angles, vals, 'o-', lw=2.2, color=colors[idx], label=label)
17 ax.axhline(y=0.80, color=CORAL, ls='--', lw=1.2, alpha=0.6, label='强鲁棒性阈值0.80')

```

H 数据预处理与验证代码

代码：附件数据加载与交叉校验（完整版）

Listing 10: 数据导入与一致性校验

```

1 # 读取5附件，9张工作表，210条记录
2 df_pop = pd.read_excel('附件1.xlsx', sheet_name='人口数据')
3 df_transfer = pd.read_excel('附件1.xlsx', sheet_name='转移概率')
4 df_demand = pd.read_excel('附件2.xlsx', sheet_name='服务需求')
5 df_revenue = pd.read_excel('附件2.xlsx', sheet_name='营收支出')
6 df_consume = pd.read_excel('附件2.xlsx', sheet_name='消费上限')
7 df_cost = pd.read_excel('附件3.xlsx', sheet_name='建设运营成本')
8 df_dist = pd.read_excel('附件4.xlsx', sheet_name='距离矩阵', index_col=0)
9 df_satisfy = pd.read_excel('附件5.xlsx', sheet_name='满意度规则')
10
11 # 交叉校验：三类老人数之和 = 60+总人口
12 assert all(df_pop['自理'] + df_pop['半失能'] + df_pop['失能'] == df_pop['60+总人口'])
13
14 # 距离矩阵对称性检验
15 assert (df_dist.values == df_dist.values.T).all(), '距离矩阵不对称!'
16
17 # 三角不等式检验（保留<200m的轻微违反为道路迂回）
18 violations = []
19 for i in range(10):
20     for j in range(10):
21         for k in range(10):
22             detour = df_dist.iloc[i,j] + df_dist.iloc[j,k] - df_dist.iloc[i,k]
23             if detour < -200:
24                 violations.append((i,j,k, -detour))
25 print(f'三角不等式显著违反: {len(violations)}处（阈值200m）')
26
27 # 缺失值检验
28 total_cells = sum(df.shape[0]*df.shape[1] for df in all_dfs)
29 missing_cells = sum(df.isnull().sum().sum() for df in all_dfs)
30 print(f'数据完整率: {(1-missing_cells/total_cells)*100:.1f}% ({total_cells}单元格)')

```

代码：距离矩阵可达性与覆盖度分析（完整版）

Listing 11: 空间可达性预分析

```

1 R_max = 1000 # 服务半径硬约束
2
3 # 各小区可达邻居数
4 reachable = (df_dist <= R_max).sum(axis=1) - 1 # 扣除自身
5 print('各小区可达邻居数:', reachable.to_dict())
6 # 输出: A:4, B:6, C:8, D:8, E:6, F:4, G:8, H:8, I:8, J:8
7
8 # 90条边中70条 <=1000m (77.8%)
9 edges_total = 10 * 9
10 edges_reachable = (df_dist.values <= R_max).sum() - 10
11 print(f'可达边占比: {edges_reachable}/{edges_total} = {edges_reachable/edges_total*100:.1f}%')
12
13 # 识别地理孤立小区 (可达邻居数最少)
14 isolated = reachable[reachable <= reachable.quantile(0.25)].index.tolist()
15 print(f'地理孤立小区 (下四分位): {isolated}') # A, F
16
17 # 空间距离矩阵热力图
18 fig, ax = plt.subplots(figsize=(10, 8))
19 sns.heatmap(df_dist, annot=True, fmt='.0f', cmap='YlOrRd',
20             xticklabels=communities, yticklabels=communities, linewidths=0.5, ax=ax)
21 ax.set_title('10小区距离矩阵 ($D_{ij}$, m)', fontsize=14, fontweight='bold')
22 fig.savefig('../figures/figure_distance_heatmap.png', dpi=300)

```

I 补充数据表

表 13: 三场景四维度韧性系数 $\rho = 1 - |\Delta X/X_{\text{baseline}}|$

场景	覆盖率韧性	满意度韧性	利润率韧性	综合韧性
A: 人口结构冲击	0.900	0.967	0.849	0.903
B: 管理成本 +20%	1.000	0.991	0.929	0.999
C: 预算调整至 140 万	1.000	0.953	0.920	0.996

表 14: Markov 链与 Leslie 矩阵双模型预测对比

指标	Markov 链	Leslie 矩阵	偏差	偏差率 (%)
第 5 年末 60+ 总人口	7,577	7,709	132	1.7
第 5 年末自理	4,981	5,063	82	1.6
第 5 年末半失能	1,471	1,502	31	2.1
第 5 年末失能	1,125	1,144	19	1.7
年均增长率 (%)	2.00	2.04	0.04	2.0
主特征值 λ_1	—	1.0198	—	—

表 15: 三场景灵敏度分析全量对比

场景	预算 (万)	选址方案	覆盖率 (%)	加权 S	年利润 (万)
基线 (Q2)	120	F(大)+H(中)+J(中)	100	0.850	1 097.7
A: 人口冲击 *	120	B(小)+D(大)+G(小)+I(中)	90.0	0.878	931.7
B: 成本 +20% [†]	120	A(中)+F(中)+G(大)	100	0.842	1 019.9
C: 预算 140 万	140	B(大)+D(大)+E(大)	100	0.889	1 010.0

* $\beta = 8\%$, $\alpha_{S \rightarrow M} = 5.5\%$, $\alpha_{M \rightarrow D} = 9.5\%$. [†] 日固定管理成本上浮 20%.